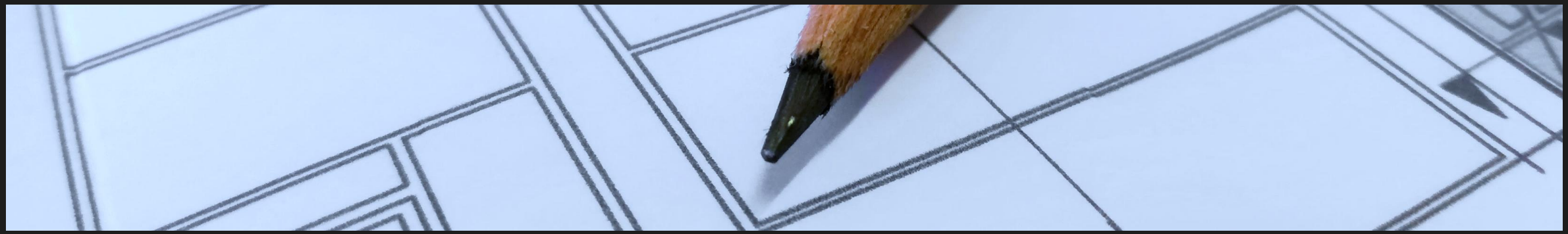
Exkurs U

Softwaremodellierung mit UML



FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Agenda

Abschnitt U.1
Einführung
Prof. Dr. Philipp Rohde 25WiSe-SWT | 328

Abschnitt U.2
Anwendungsfalldiagramme
Prof. Dr. Philipp Rohde 25WiSe-SWT | 335

Abschnitt U.3
Klassen- & Objektdiagramme
Prof. Dr. Philipp Rohde 25WiSe-SWT | 349

Abschnitt U.4
Aktivitätsdiagramme
Prof. Dr. Philipp Rohde 25WiSe-SWT | 369

Abschnitt U.5
Zustandsdiagramme
Prof. Dr. Philipp Rohde 25WiSe-SWT | 389

Abschnitt U.6
Sequenzdiagramme
Prof. Dr. Philipp Rohde 25WiSe-SWT | 403

Klausurrelevante Lernziele

- Unterschied zwischen Struktur- und Verhaltensdiagrammen erläutern, Beispiele für zugehörige UML-Diagramme nennen können
- Anwendungsfall-, Klassen-, Aktivitäts-, Zustands- & Sequenzdiagramme
  - Einsatzgebiet, Zweck & relevante Phasen erläutern können
  - Gegebene Diagramme interpretieren können
  - Syntaktisch korrekte, einfache Diagramme auf Grundlage einer textuellen Beschreibung erstellen können
- Nicht klausurrelevant
  - Anwendungsfalldiagramme: Multiplizitäten von Assoziationen, Generalisierung & Spezialisierung
  - Klassendiagramme: parametrisierte Klassen (Generics), Aufzählungen (Enumerations), Assoziationsklassen, Abhängigkeiten außer «use»
  - Aktivitätsdiagramme: zweidimensionale Aktivitätsbereiche, Vor- & Nachbedingungen von Aktivitäten
  - Sequenzdiagramme: andere kombinierte Fragmente außer »alt«

Option: Zertifizierung

- OMG Certified UML Professional 2 (OCUP2)
- Weltweit anerkanntes Zertifikat für die korrekte Interpretation und Konstruktion von UML-Diagrammen (Version 2.5)
  - Durch: Object Management Group (OMG)
  - Level: Foundation, Intermediate, Advanced
  - Foundation: »It covers the UML elements and constructs used most heavily to read or construct UML models of systems worked by teams in business today. The diagram types covered at this level – Class, Object, and Package on the structural side; Sequence, Activity, State Machine, and Use Case on the behavioral – depict the features that architects and developers need to visualize as they design and implement real, practical, everyday systems […]«
- Prüfung bei unabhängigem Zertifizierer
  - Multiple Choice, 90 Fragen, 150 Minuten (»non-natives«), Bestehensquote: 66%
  - Sprache: nur englisch
  - Gebühr: $350 plus MwSt.; Rabatte möglich; Zertifikat ist 3 Jahre gültig
  - Online-Prüfung möglich, Online-Beispielprüfung
- Vorlesung bereitet weitgehend auf die Prüfung vor

Abschnitt U.1

Einführung

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Modellierung (1)

- Modelle: Abstraktion & Abbildung eines realen Systems
  - Vereinfachung & gezieltes Weglassen von Details
  - Wesentliche Strukturen erkennbar machen
  - Fokus auf den jeweiligen Zweck des Modells
  - Für existierendes oder zu erstellendes System
- Systemtheorie (aka Kybernetik)
  - Statische Modellierung: beschreibt Struktur eines Systems
  - Dynamische Modellierung: beschreibt Verhalten eines Systems
- Modelle ermöglichen bzw. erleichtern
  - … gemeinsames Verständnis
  - … Kommunikation der Stakeholder untereinander
  - … Spezifikation
  - … Analyse
  - … Simulation
  - … Erzeugung eines Grundgerüsts

Modellierung (2)

- Grundsätze ordnungsmäßiger Modellierung (GoM)
  (1) Grundsatz der Richtigkeit
  (2) Grundsatz der Relevanz
  (3) Grundsatz der Wirtschaftlichkeit
  (4) Grundsatz der Klarheit
  (5) Grundsatz der Vergleichbarkeit
  (6) Grundsatz des systematischen Aufbaus
- Modellierung in der Softwaretechnik
  - Modellierung des Systemkontextes
  - Modellierung von Anforderungen & fachlicher Zusammenhänge
  - Modellierung von funktionalen Aspekten einer Software
    - Struktur & Daten
    - Funktion & Ablauf
    - Zustand & Verhalten
  - Modellierung statischer & dynamischer Codestrukturen
  - Modellierung zur Codegenerierung

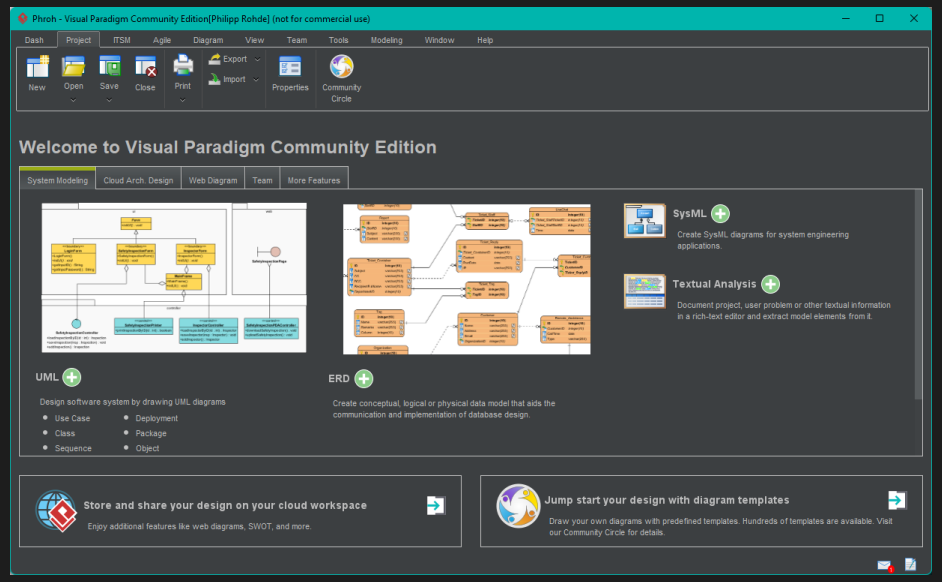
Quelle: (Becker, Probandt, Vering 2012)

UML: Historie

- Entwicklung der Unified Modeling Language (UML)
  - Zusammen mit Object-Oriented Software Engineering (OOSE) [→ Kapitel 4]
  - Seit 1997 Object Management Group (OMG); ISO-Norm seit 2012
  - UML 1.0 (1997), UML 2.0 (2005), UML 2.5.1 (Dezember 2017)
- Visualisierung, Spezifikation, Konstruktion und Dokumentation von objekt-orientierten Modellen und Entwürfen
- UML ist unabhängig von der (objektorientierten) Programmiersprache
  - Konstruktionen, die in den meisten OO-Sprachen vorhanden sind
  - Dadurch sind aber spezifische Konstruktionen ggf. nicht möglich
- Standardisiertes UML folgt weitgehend der Java-Notation
  - Klassen und Schnittstellen in Pascal Case, z.B. MotorVehicle
  - Attribute, Operationen, Namensräume in Camel Case, z.B. getSpeed
- Primitive UML-Datentypen: Boolean, Integer, Real, String
  - Es ist sinnvoll, die primitiven Datentypen der konkreten Zielsprache zu verwenden, z.B. Java oder C#

Quelle: (ISO/IEC 19505-1:2012; Object Management Group 2017)

UML-Werkzeug



- Empfehlung: Visual Paradigm
  - https://www.visual-paradigm.com
  - Kostenlose Community Edition (mit Registrierung)
  - Beachte: Window » Configure Programming Language auf Java stellen
- Alternativen
  - Gaphor (https://gaphor.org)
  - Draw.io (im Browser, eingeschränktes UML, https://www.drawio.com)
  - PlantUML (https://plantuml.com)

UML-Diagramme: Strukturdiagramme

- Sieben Strukturdiagramme (Structure Diagrams)
- Ziel: Modellierung der statischen Aspekte eines Systems
- Arten:
  - Paketdiagramm (Package Diagram)
  - Komponentendiagramm (Component Diagram)
  - Kompositionsstrukturdiagramm (Composite Structure Diagram)
  - Klassendiagramm (Class Diagram)
  - Objektdiagramm (Object Diagram)
  - Verteilungsdiagramm (Deployment Diagram)
  - Profildiagramm (Profile Diagram)

UML-Diagramme: Verhaltensdiagramme

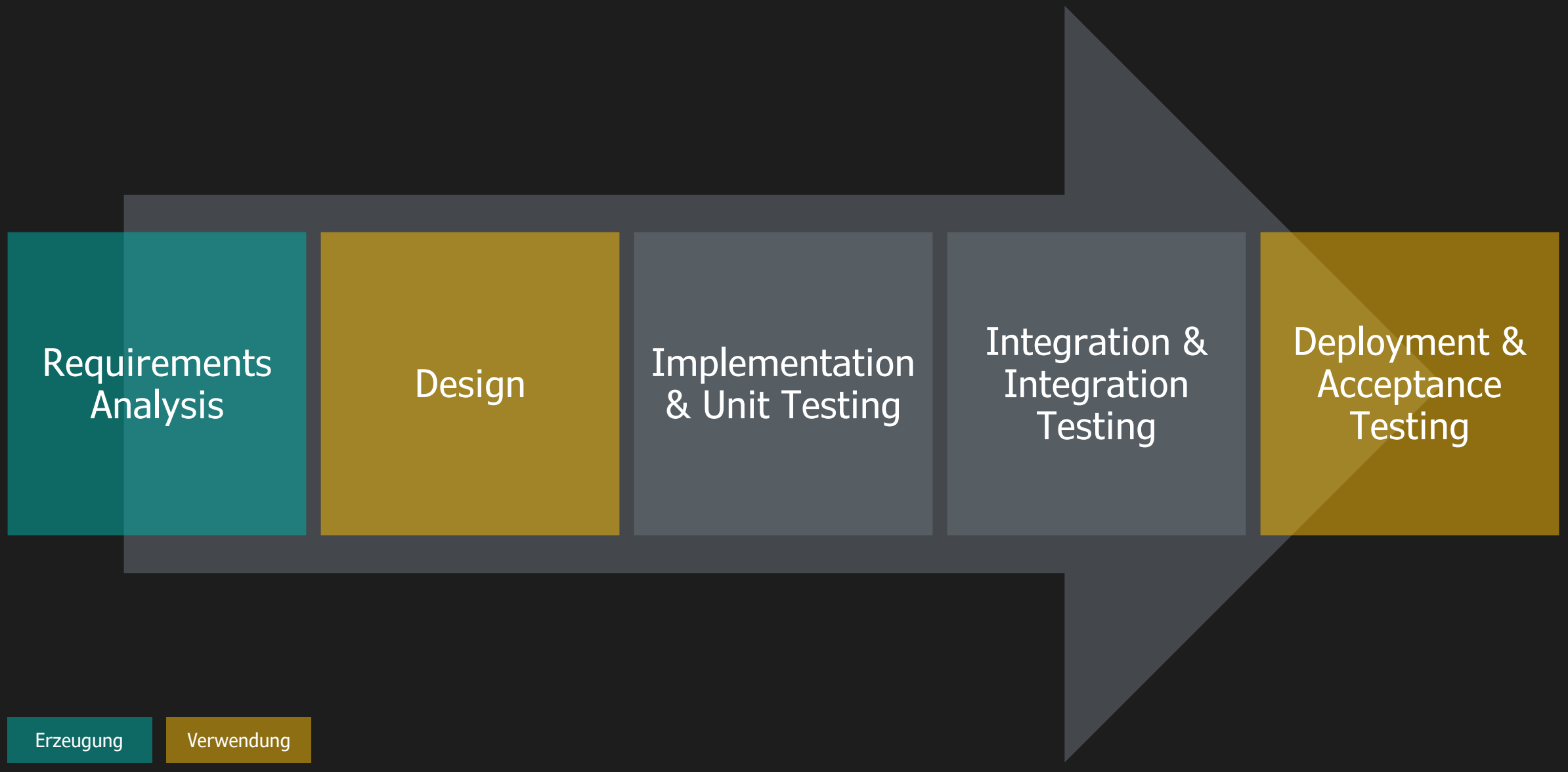
- Sieben Verhaltensdiagramme (Behavior Diagrams)
- Ziel: Modellierung der dynamischen Aspekte eines Systems
- Arten
  - Anwendungsfalldiagramm (Use Case Diagram)
  - Aktivitätsdiagramm (Activity Diagram)
  - Zustandsdiagramm (State Machine Diagram)
  - Interaktionsdiagramme (Interaction Diagrams)
    - Sequenzdiagramm (Sequence Diagram)
    - Kommunikationsdiagramm (Communication Diagram)
    - Interaktionsübersichtsdiagramm (Interaction Overview Diagram)
    - Zeitdiagramm (Timing Diagram)

Abschnitt U.2

Anwendungsfalldiagramme

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Relevante Phasen der Softwareentwicklung



Motivation

Use Case – (1) In UML, a complete task of a system that provides a measurable result of value for an actor.

(2) Sequence of tasks that a system can perform, interacting with users of the system and providing a measurable result of value for the user.

(3) Description of the behavioral requirements of a system and its interaction with a user.

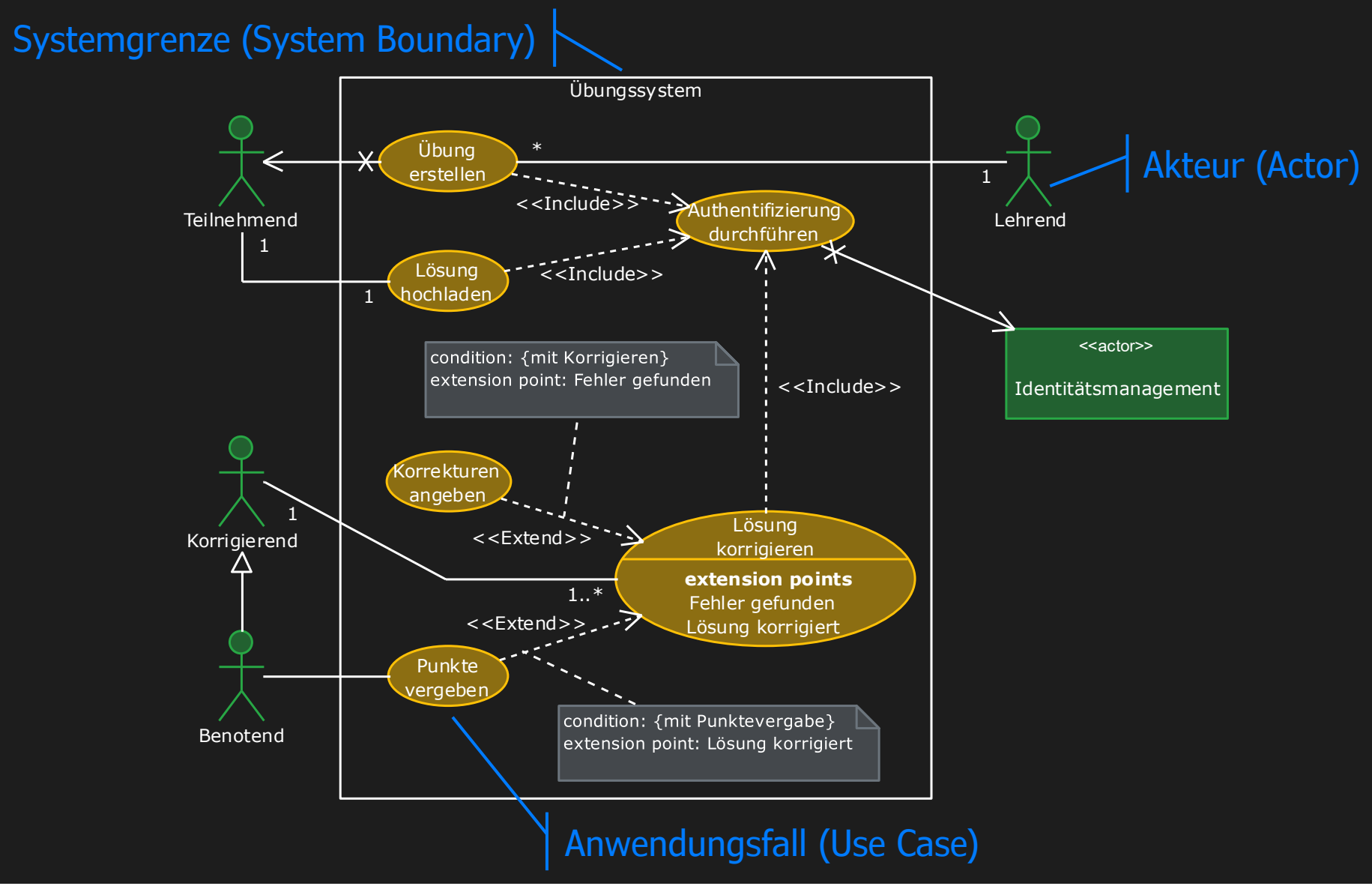
(ISO/IEC/IEEE 24765:2017)

- Anwendungsfall
  - Beschreibt die Interaktion eines / mehrerer Akteure mit einem System
  - Außenansicht: nur extern »wahrnehmbare« Funktionalität
  - Beschreibt nur, was das System für Funktionalität anbietet und was ein Akteur erwarten kann; aber nicht, wie es realisiert ist
  - Immer mit erkennbarem Nutzen / Ergebnis für mindestens einen Akteur

Anwendungsfalldiagramm

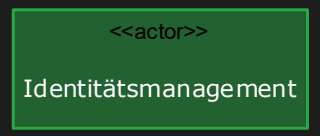
- Bewusst einfach gehalten; nur wenige Elemente
- Umfasst immer ein System & typischerweise mehrere Anwendungsfälle
- Keine Modellierung der Reihenfolge der Interaktion (Prozessablauf); keine Parameter oder Rückgaben
- Einsatz überwiegend in der Phase der Requirements Analysis: Modellierung der Bedürfnisse der Stakeholder
- Bildet die Basis für die weitere Klärung & Spezifikation, z.B. mit UML-Verhaltensdiagrammen
- Bildet auch Grundlage für spezifische Tests, insbesondere Akzeptanz- bzw. Abnahmetests (User Acceptance Tests, UAT)
- Vollständiger Anwendungsfall durch textuelle Erweiterung, um Vor- und Nachbedingungen, Regelablauf, Ausnahmefälle etc. zu beschreiben

Beispiel: Übungssystem



Systemgrenze & Akteure

- Systemgrenze
  - Begrenzt das System, das die Anwendungsfälle bereitstellt und mit dem die Nutzenden interagieren
  - Muss immer angegeben werden
- Akteur
  - Rolle, die externe Nutzende oder Systeme während der Interaktion einnehmen
  - Akteure immer außerhalb der Systemgrenze
  - Personen können ggf. verschiedene Rollen annehmen, z.B. Teilnehmend und Korrigierend
  - Personen immer als Strichfigürchen (Stickman)
  - Eigene Symbole sind erlaubt
  - Generalisierung (analog zu Klassendiagramm): die Spezialisierung »erbt« alle Anwendungsfälle der Generalisierung



Anwendungsfälle

Anwendungsfall

Alternative Darstellung

- Anwendungsfall
  - Abgeschlossene Menge von Aktionen, die bereitgestellt werden
  - Immer innerhalb der Systemgrenze, die diese bereitstellt
  - Die genaue Reihenfolge der Aktionen wird nicht spezifiziert
  - Namenskonvention: »Substantiv + Verb«, z.B. »Übung erstellen«
- Regeln
  - Anwendungsfälle werden immer von Akteuren ausgelöst
  - Zusammenfassung aller Aktionen, die aus Sicht des Akteurs zu einem »wahrnehmbaren« Ergebnis führen
  - Nicht zu viele Details: nur Grundlage für die weitere Spezifikation
  - Eine zu »technische« Sicht sollte vermieden werden
  - Vollständiger Anwendungsfall besitzt ergänzende textuelle Beschreibung (siehe unten)

Assoziationen

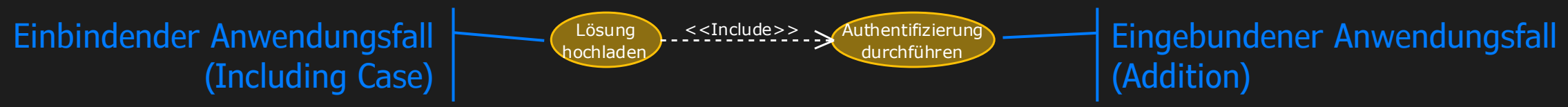
- Beziehung zwischen Akteur und Anwendungsfall
- Multiplizitäten & Navigierbarkeit wie bei Klassendiagrammen, z.B. 0..1 (höchstens eins), 1 (genau eins), 1..* (mindestens eins), * (beliebig)
- Ungerichtet: Akteur ist beteiligt, kann auslösen oder Informationen erhalten
- Gerichtet: Akteur ist am Anwendungsfall beteiligt, kann diesen aber nicht auslösen

Jede teilnehmende Person kann genau eine Lösung hochladen und jede Lösung gehört zu genau einer teilnehmenden Person

Jede korrigierende Person kann mehrere Lösungen korrigieren und jede Korrektur gehört zu genau einer korrigierenden Person

Teilnehmer sind beteiligt (z.B. »wird benachrichtigt«), können aber keine Übung erstellen

Include-Beziehung



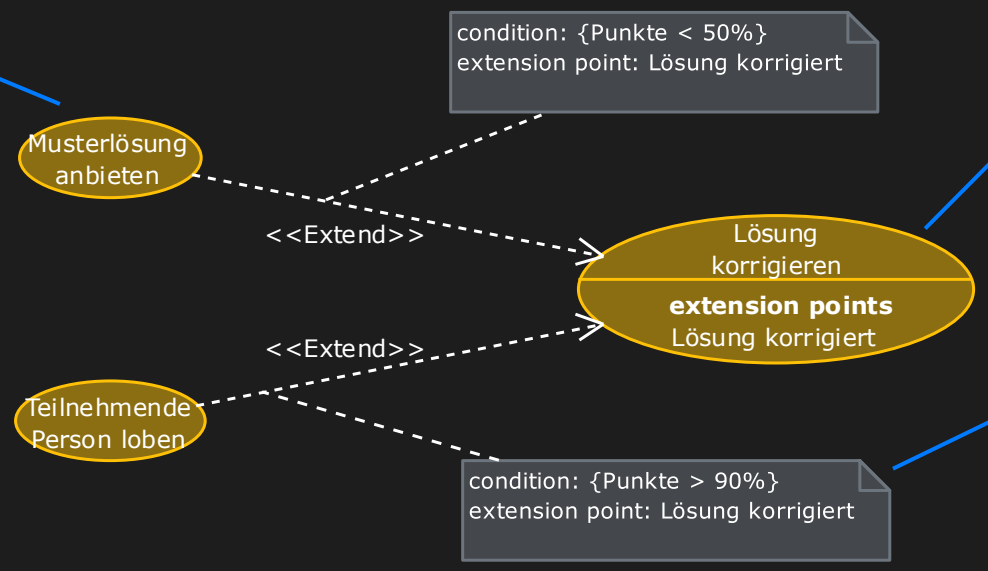
- Unbedingte Einbindung eines Anwendungsfalls vor Ausführung eines anderen Anwendungsfalls
- Immer wenn der einbindende Anwendungsfall ausgeführt wird, dann wird vorher auch der eingebundene Anwendungsfall ausgeführt
- Einbindender Anwendungsfall ist abhängig vom Ergebnis des eingebundenen Anwendungsfalls
- Sinnvoll, um Aktionen wiederzuverwenden
- Ketten sind möglich, aber Zyklen müssen vermieden werden

Extend-Beziehung

- Bedingte Einbindung eines Anwendungsfalls während der Ausführung eines anderen Anwendungsfalls
  - Bedingung & Erweiterungspunkt als Kommentar an der Extend-Beziehung
  - Erweiterungspunkt ist Zeitpunkt bzw. Ereignis während der Ausführung des erweiterten Anwendungsfalls
  - Wenn Erweiterungspunkt erreicht wird und zu diesem Zeitpunkt die Bedingung erfüllt ist, dann wird auch der erweiternde Anwendungsfall ausgeführt
- Beachte
  - Alle möglichen Erweiterungspunkte werden im erweiterten Anwendungsfall unterhalb von »extension points« angezeigt
  - Mehrere erweiternde Anwendungsfälle können sich auf den gleichen Erweiterungspunkt beziehen
  - Wenn keine Bedingung angegeben ist, dann wird der erweiternde Anwendungsfall immer ausgeführt
  - Ketten sind möglich, aber Zyklen müssen vermieden werden

Beispiel: Extend

Erweiternder Anwendungsfall (Extension)

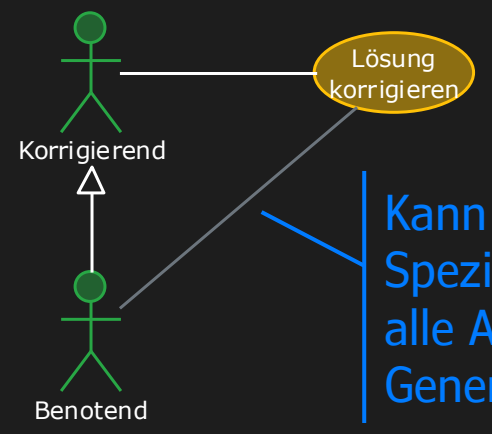


Erweiterter Anwendungsfall (Extended Case) mit allen Erweiterungspunkten (Extension Points)

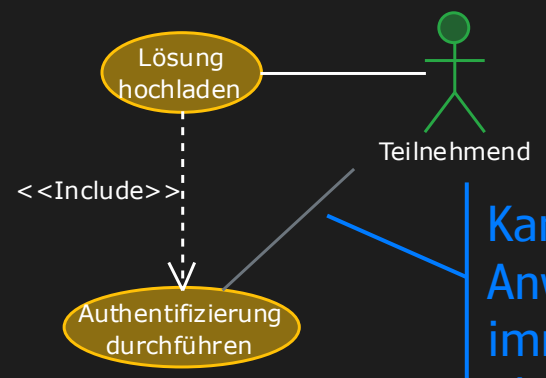
Bedingung (Condition)

Angabe des Erweiterungspunkts des erweiterten Anwendungsfalls, an dem der erweiternde Anwendungsfall eingebunden wird

Konvention: implizite Assoziationen

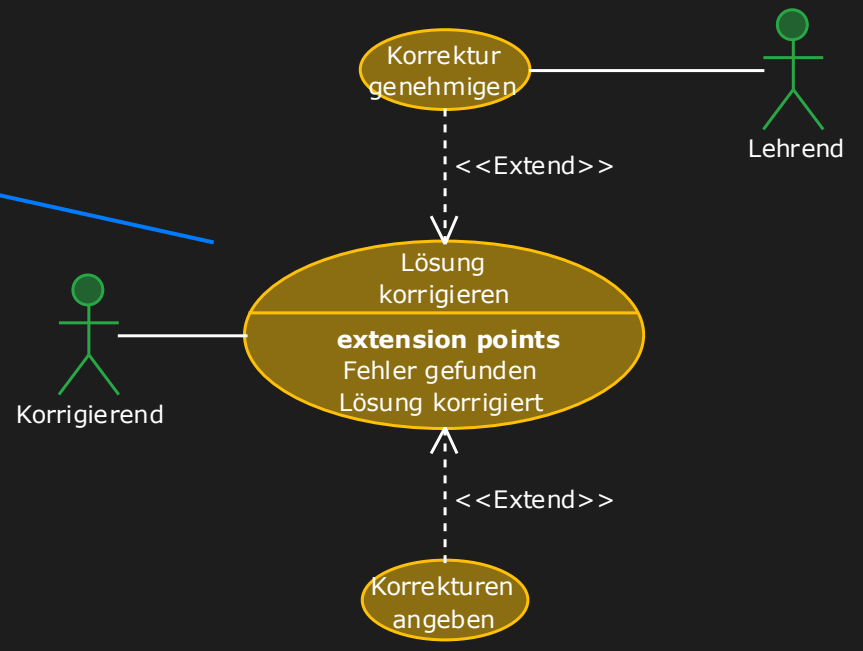


Kann wegfallen: Spezialisierungen erben alle Anwendungsfälle der Generalisierung



Kann wegfallen: Eingebundene Anwendungsfälle werden immer von den gleichen Akteuren ausgeführt

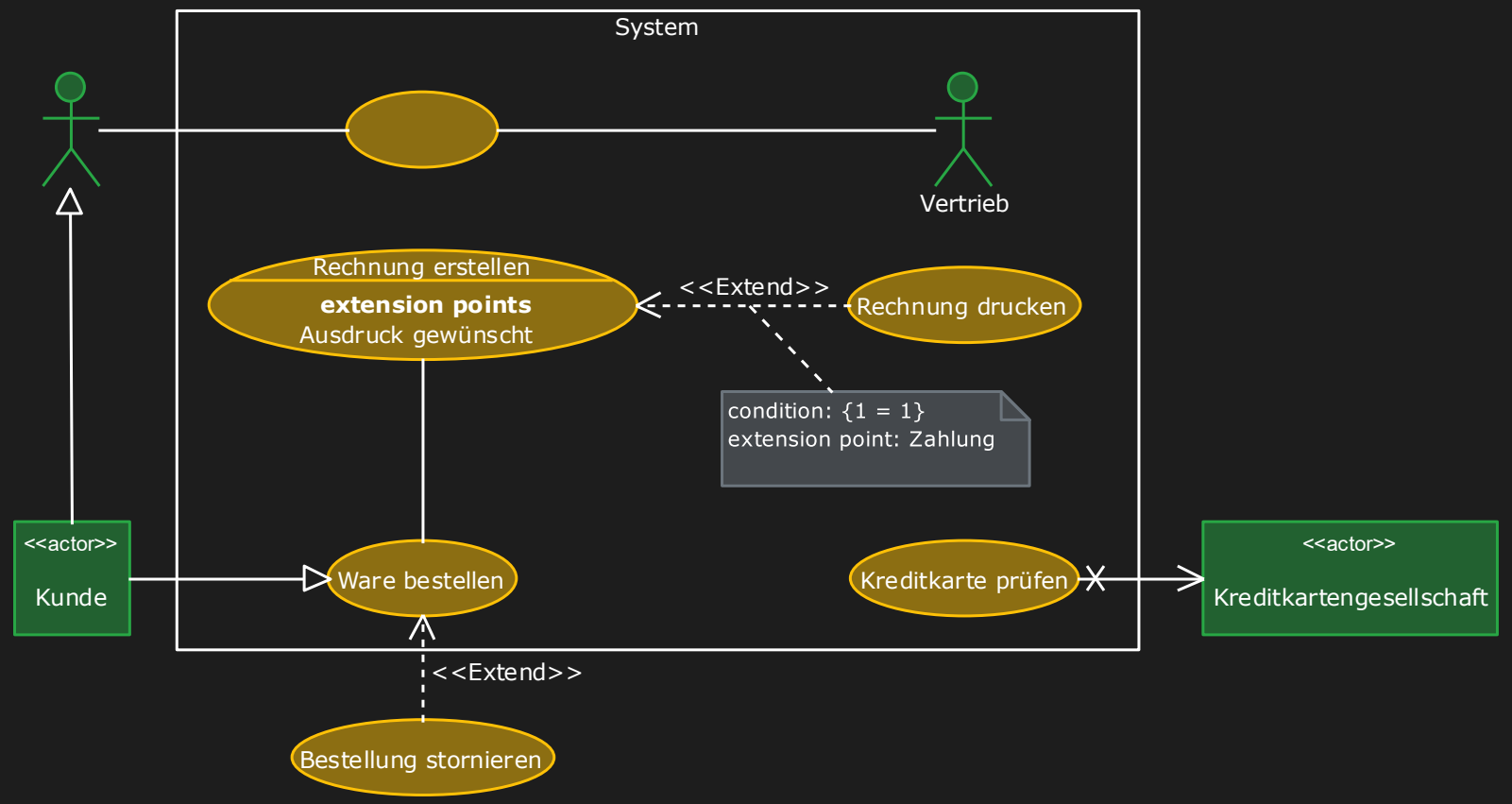
- Explizite Angabe hat Priorität: Lehrende dürfen Korrektur genehmigen, Korrigierende dürfen es nicht
- Erweiterte Anwendungsfälle erben Assoziationen von erweiternden Anwendungsfällen: Lehrende dürfen auch Lösung korrigieren
- Keine Angabe von Assoziationen bei erweiternden Anwendungsfällen: alle Assoziationen des erweiterten Anwendungsfall werden vererbt, d.h. Korrigierende und Lehrende dürfen Korrekturen angeben



Beispiel: Vollständiger Anwendungsfall

Use Case	Lösung hochladen
Story & Justification	»Als teilnehmende Person möchte ich eine Lösung zu einer Übung im Übungssystem hochladen, damit diese korrigiert werden kann«
Primary Actors	Teilnehmende Person
Supporting Actors	Identitätsmanagement
Pre-Conditions	▪ Teilnehmende Person ist authentifiziert ▪ Teilnehmende Person ist bei zugehöriger Vorlesung angemeldet
Post-Conditions	Die Lösung ist im Übungssystem hinterlegt und der teilnehmenden Person zugeordnet
Flow of Events	1. Teilnehmende Person meldet sich im Übungssystem an 2. Teilnehmende Person ruft die aktuelle Übung ab 3. Teilnehmende Person eine Lösung als Dokument hoch (PDF oder Bild) 4. Der Ladevorgang wird erfolgreich abgeschlossen
Exceptions	▪ Abgabefrist ist überschritten ▪ Es existiert bereits Lösung der teilnehmenden Person für diese Übung ▪ Lösungsdokument hat falsches Dateiformat oder ist zu groß ▪ Ladevorgang schlägt fehl oder wird abgebrochen

Anwendungsfalldiagramm: 12 typische Fehler

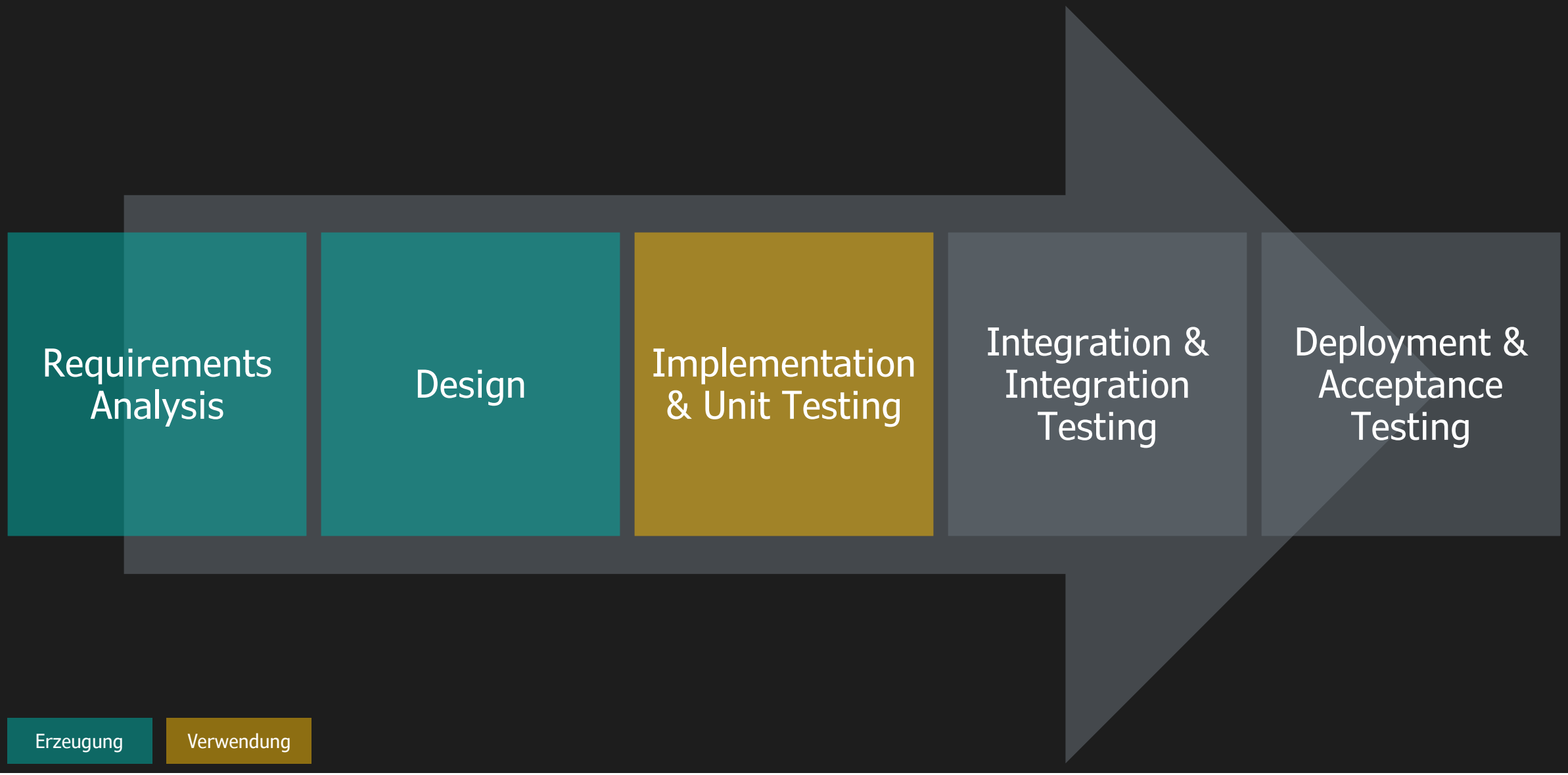


Abschnitt U.3

Klassen- & Objektdiagramme

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Relevante Phasen der Softwareentwicklung

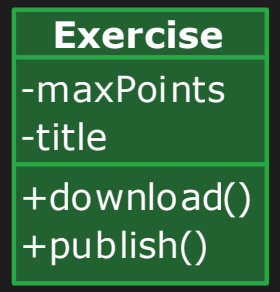


Motivation

- Modellierung von statischen Strukturen
  - Klassen: »Bauplan« für Objekte mit gleicher Datenstruktur (Attribute) & Verhalten (Operationen)
  - Assoziationen: Beziehungen zwischen Klassen
  - Vererbung: Generalisierung bzw. Spezialisierung
- Fortlaufende Verfeinerung
  - Requirements Analysis: nur Hauptobjekte und deren Beziehungen, nur öffentliche Attribute & Operationen, keine Details (Datentypen, Parameter etc.), bewusst einfache Diagramme
  - Ziel: Klärung von Korrektheit und Vollständigkeit mit Stakeholdern
  - Entwurf: Vorbereitung der (verteilten) Implementierung, Ergänzung von Hilfsklassen & internen Attributen, Operationen & Beziehungen, Spezifikation der Details (Datentypen, Parameter etc.)
- Beachte: hier nur die wichtigsten Konstruktionselemente

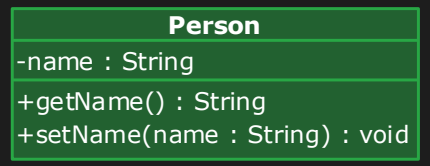
Klassen (1)

- Dreiteilige Darstellung einer Klasse mit Klassenmitgliedern
  - Klassenname: immer Singular
  - Attribute: Felder einer Klasse mit Sichtbarkeit
  - Operationen: Methoden einer Klasse mit Sichtbarkeit
- Sichtbarkeit von Attributen und Operationen
  - Private (-) Zugriff nur durch die Klasse selbst
  - Package (~) Zugriff durch Klassen im gleichen Paket
  - Protected (#) Zugriff auch durch abgeleitete Klassen
  - Public (+) öffentlicher Zugriff durch jede andere Klasse
- Beachte: die Sichtbarkeit der Klasse selbst wird nicht dargestellt
- Abstrakte Klassen & Klassenmitglieder: *kursiv* oder Angabe von {abstract} (bei handschriftlichen Diagrammen)
- Statische Klassenmitglieder: <u>unterstrichen</u>
- Reduzierte Anzeige ohne Klassenmitglieder möglich

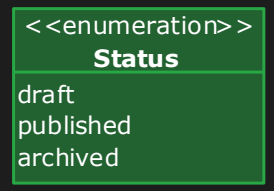


Klassen (2)

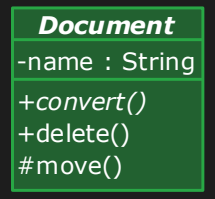
Reduzierte Klasse



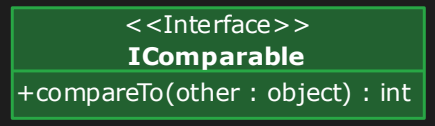
Attribut mit Datentyp und Getter & Setter



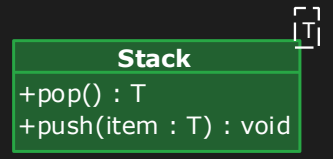
Aufzählung (Enumeration)



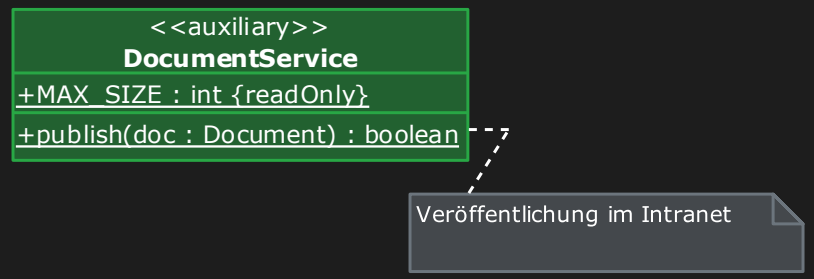
Abstrakte Klasse mit abstrakter Operation (alternativ: {abstract})



Schnittstelle (Interface)



Parametrisierte Klasse (Template)

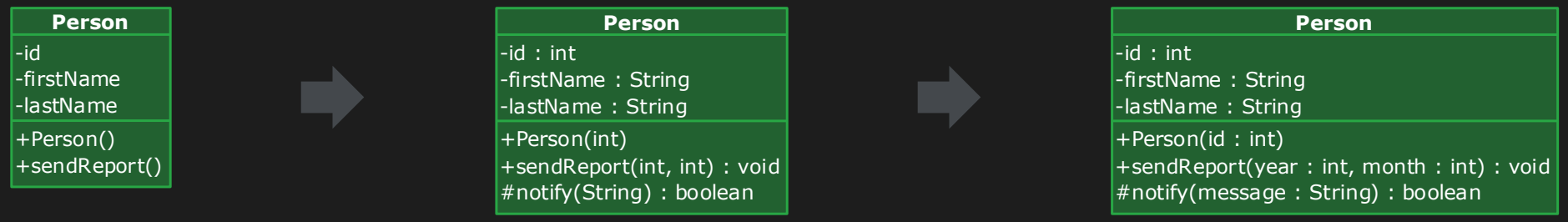


Anmerkung mit Zuordnung zu Operation

Klasse mit Stereotyp «auxiliary», öffentlicher Konstante & statischer Operation

Datentypen & Detailstufen

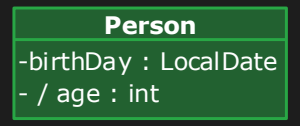
- Angabe des Datentyps
  - Hinter Attributen, Parametern bzw. Operationen
  - Trennung durch Doppelpunkt
- Konvention: Detailstufen bei der Modellierung
  - Start: Attribute & öffentliche Operationen ohne Parameter / Datentypen
  - Zwischenstufe: Datentypen von Attributen, Anzahl & Datentypen der Parameter, Rückgabe-datentyp von Operationen, nicht-öffentliche Operationen
  - Final: Vollständige Modellierung gemäß UML-Spezifikation, z.B. Parameternamen



Start Zwischenstufe Final

Attribute & Operationen (1)

- Attribute
  - In OOP sollten Attribute immer privat sein
  - Lesen & Setzen der Werte durch Getter- & Setter-Operationen
  - Ausnahme: öffentliche Konstanten als statische ReadOnly-Attribute
  - Abgeleitete Attribute werden durch vorangestelltes / markiert
    - Werte werden bei Abruf berechnet und nicht gespeichert, z.B. »Alter« als Differenz von »Heute« und Geburtsdatum (Java-Implementierung als Operation)



- Operationen
  - Fehlende Angabe des Rückgabetyp bedeutet »unbekannt«
  - Für Operationen ohne Rückgabewert (aka Prozedur) muss explizit »void« als Rückgabetyp angegeben werden
  - Ausnahme: Konstruktoren, siehe unten
- Multiplizitäten: Angabe durch Intervall »lower..upper«
  - Unbestimmt: * (äquivalent zu 0..*)
  - Wertebereich, z.B. 0..2 oder 1..*
  - Wert, z.B. 4 (äquivalent zu 4..4)

Attribute & Operationen (2)

- In (vereinfachter) Backus-Naur-Form (BNF)

  attribute ::= [visibility] [/] name [: type] [[multiplicity]] [= default] [{modifier[, modifier]*}]

  operation ::= [visibility] name ([parameter-list]) [: [return-type] [[multiplicity]] [{property[, property]*}]]

  parameter-list ::= parameter[, parameter]*

  parameter ::= [direction] name : type [[multiplicity]] [= default] [{property[, property]*}]

  - default: Vorgabewert oder Formel
  - modifier: z.B. readOnly, ordered, unique und id
  - property: z.B. ordered, unique
  - direction: in, out, inout (in, falls nicht angegeben)

Konventionen: Klassenmitglieder

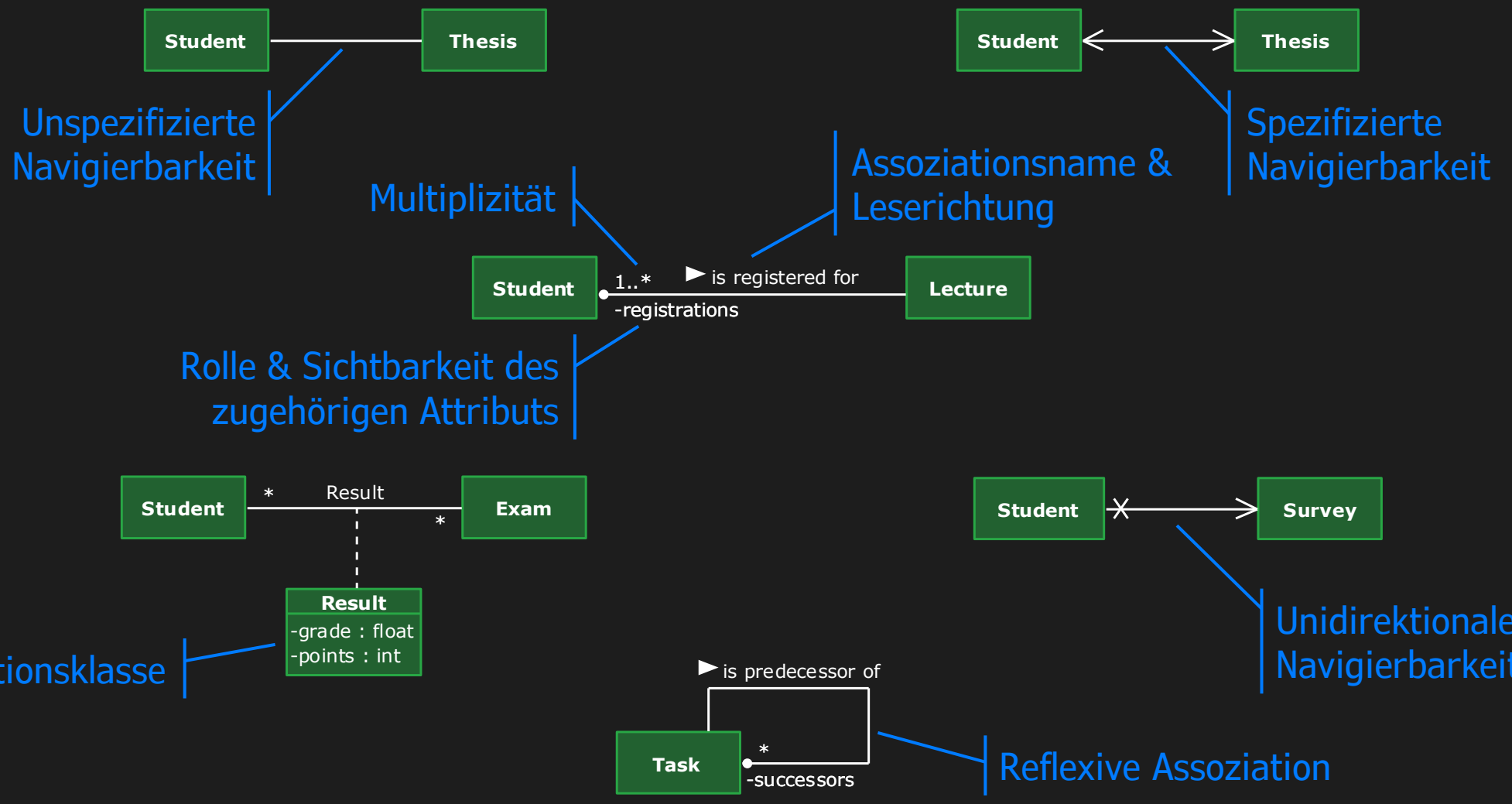
- Konstruktoren
  - Operation mit Klassennamen als Name, ohne Rückgabetyp
  - Ohne Angabe: immer öffentlicher, parameterloser Konstruktor
- Attribute
  - Ohne Angabe: normale Attribute besitzen öffentliche Getter & Setter
  - Mehrwertige Attribute (z.B. Arrays): ohne öffentliche Getter & Setter
  - {readOnly} als Modifizierer: öffentliche Getter & private Setter
  - Abgeleitete Attribute (mit /): öffentliche Getter & keine Setter
- Konstanten: Großbuchstaben (wie Java), unterstrichen (statisch) und {readOnly}
- Abstrakte Klassen
  - Ohne Angabe: abgeleitete konkrete Klasse implementiert alle abstrakten Operationen
- Interfaces
  - Nur Operationen, keine Attribute; Operationen werden nicht-kursiv dargestellt
  - Ohne Angabe: realisierende Klasse implementiert alle Operationen
- Explizite Angaben haben immer Priorität

Beispiel: Implementierung in Java

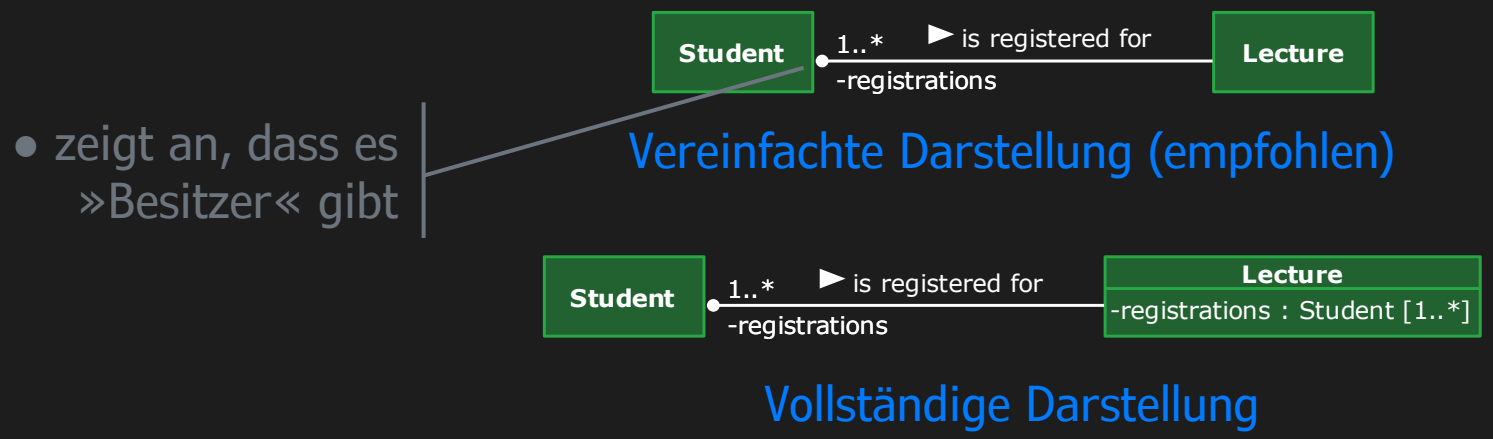
```java
class Car {
    public static final double RESERVE = 10.0;
    private double fuelLevel;
    private String name = "My Car";
    private Person[] passengers = new Person[4];
    private int speed;

    public Car(Person driver) {
        if (driver == null) { throw new NullPointerException(); }
        this.passengers[0] = driver;
    }

    public double getFuelLevel() { return fuelLevel; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getSpeed() { return speed; }
    public boolean isWarning() { return fuelLevel < RESERVE; }
    public void accelerate(int delta) { throw new UnsupportedOperationException(); }
    public void brake() { throw new UnsupportedOperationException(); }
    public void refuel(double amount) { throw new UnsupportedOperationException(); }
    protected void setFuelLevel(double fuelLevel) { this.fuelLevel = fuelLevel; }
    private boolean resetWarnings() { throw new UnsupportedOperationException(); }
}
```

Car
+RESERVE : double = 10.0 {readOnly} -fuelLevel : double -name : String = "My Car" -passengers : Person[] [1..4] -speed : int {readOnly} - / warning : boolean = (fuelLevel < reserve)
+Car(driver : Person) #setFuelLevel(fuelLevel : boolean) : void +accelerate(delta : int) : void +brake() : void +refuel(amount : double) : void -resetWarnings() : boolean

Assoziationen (1)



Assoziationen (2)

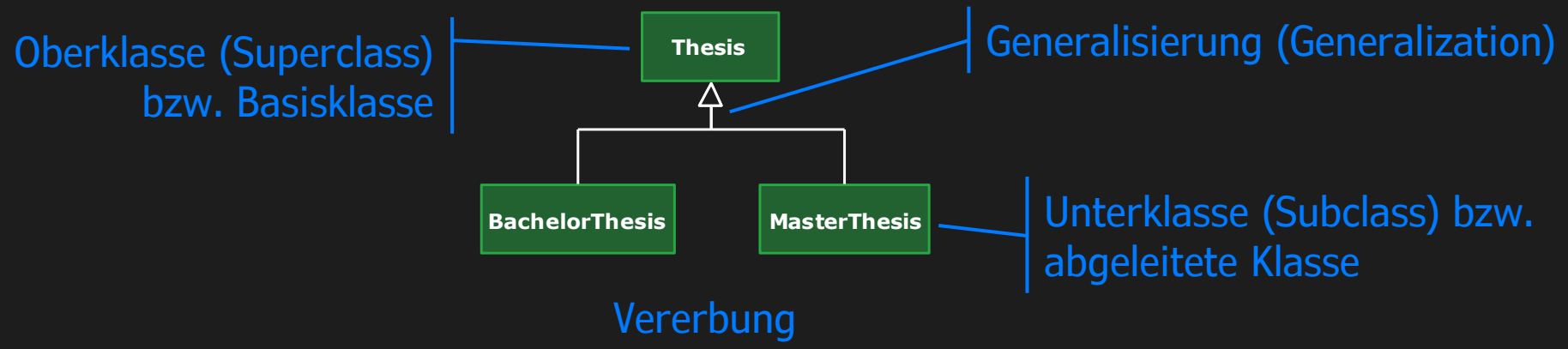


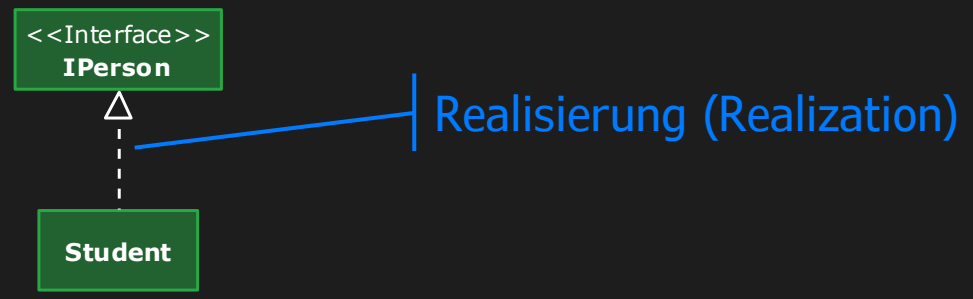
- Implementierung durch Attribute
  - Name des Attributs = Rollenname, Sichtbarkeit wie angegeben
  - Datentyp = Klasse, an der der Rollenname steht
  - Mehrfache Multiplizität: Array oder anderer Collection-Datentyp
  - Achtung: bei Multiplizität 1 bzw. 1..x muss codeseitig sichergestellt werden, dass das Attribut bzw. das erste Element zu jeder Zeit einen Wert ungleich Null hat
- Konventionen
  - Immer mindestens Angabe von Assoziationsname & Leserichtung oder Rolle
  - Möglichst immer Assoziationen nutzen (statt nur Attribute)

Qualifizierte Assoziationen

- Qualifizierer definiert Menge an Attributen, die die Objekte der (gegenüberliegen-den) Klasse eindeutig referenzieren
- Entspricht Fremdschlüssel-Konzept in Datenbanken

Vererbung & Realisierung

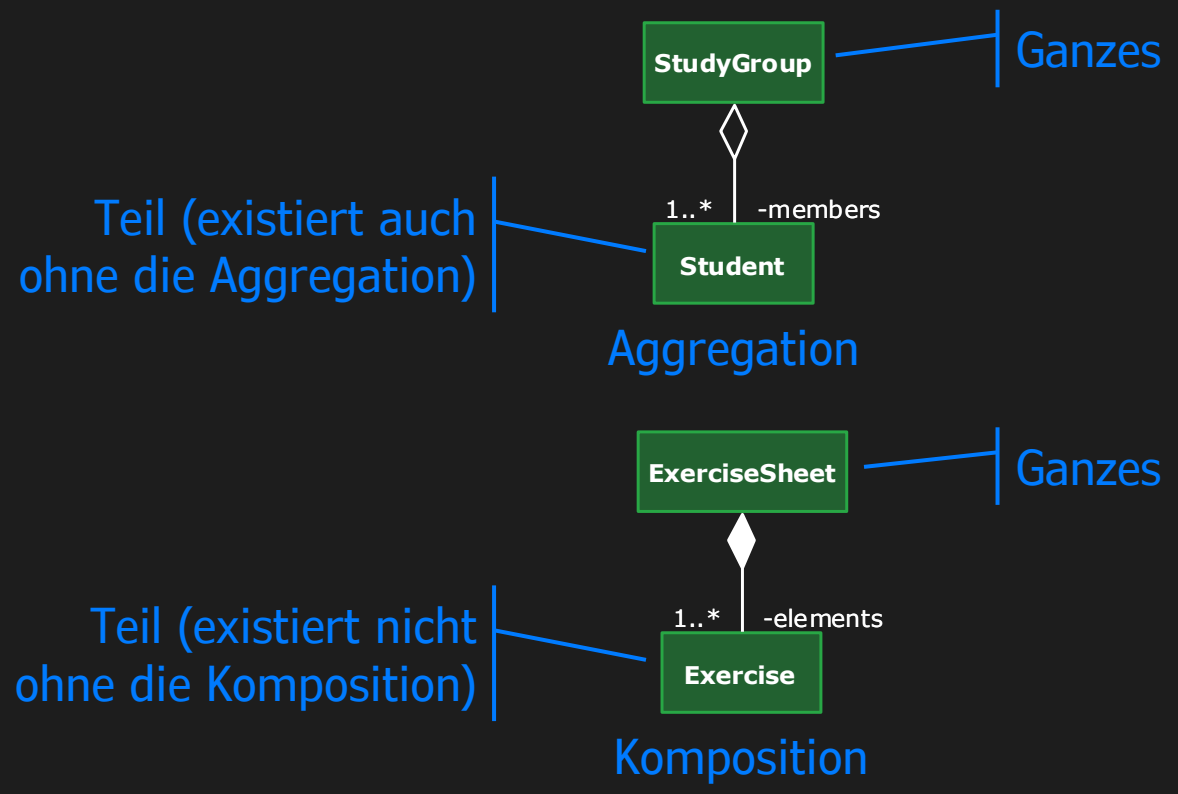




Realisierung einer Schnittstelle (Interface)

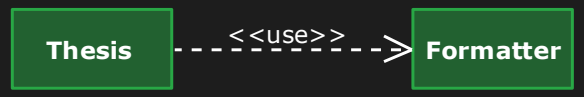
- Beachte: Pfeil zeigt immer in Vererbungshierarchie »nach oben« (Kovarianz)

Ganzes-Teile-Beziehungen



- Multiplizität 1 beim »Ganzen« kann weggelassen werden
- Komposition
  - Multiplizität beim »Ganzen« ist immer 1 (kann daher immer weggelassen werden)
  - Beim Erzeugen / Löschen des Ganzen werden auch alle Teile erzeugt / gelöscht; das Ganze kann nicht ohne die Teile, die Teile nicht ohne das Ganze existieren (Datenbanken: Löschweitergabe)

Abhängigkeiten

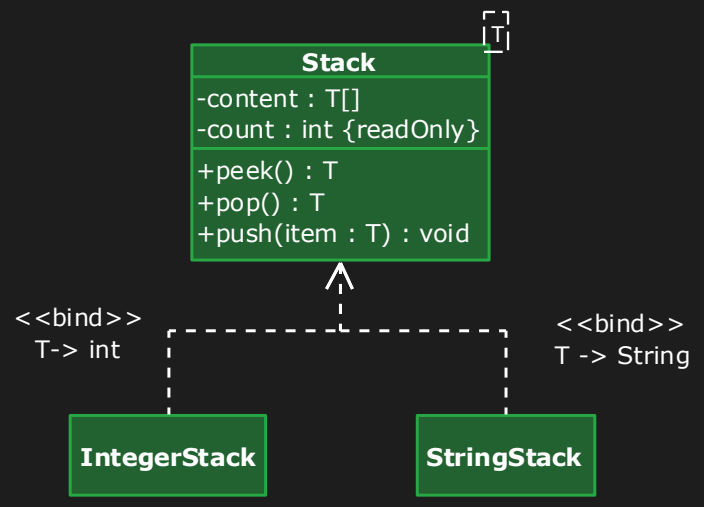


Abhängigkeit

- Allgemeine Abhängigkeit zwischen Elementen, z.B. Aufruf von Operationen, Datentyp von internen Variablen, zwischen Paketen etc.
- Spezifikation durch Stereotyp «use»
- Weitere Stereotypen, z.B. «call», «create», «derive», «instantiate», «refine»

Parametrisierten Klassen

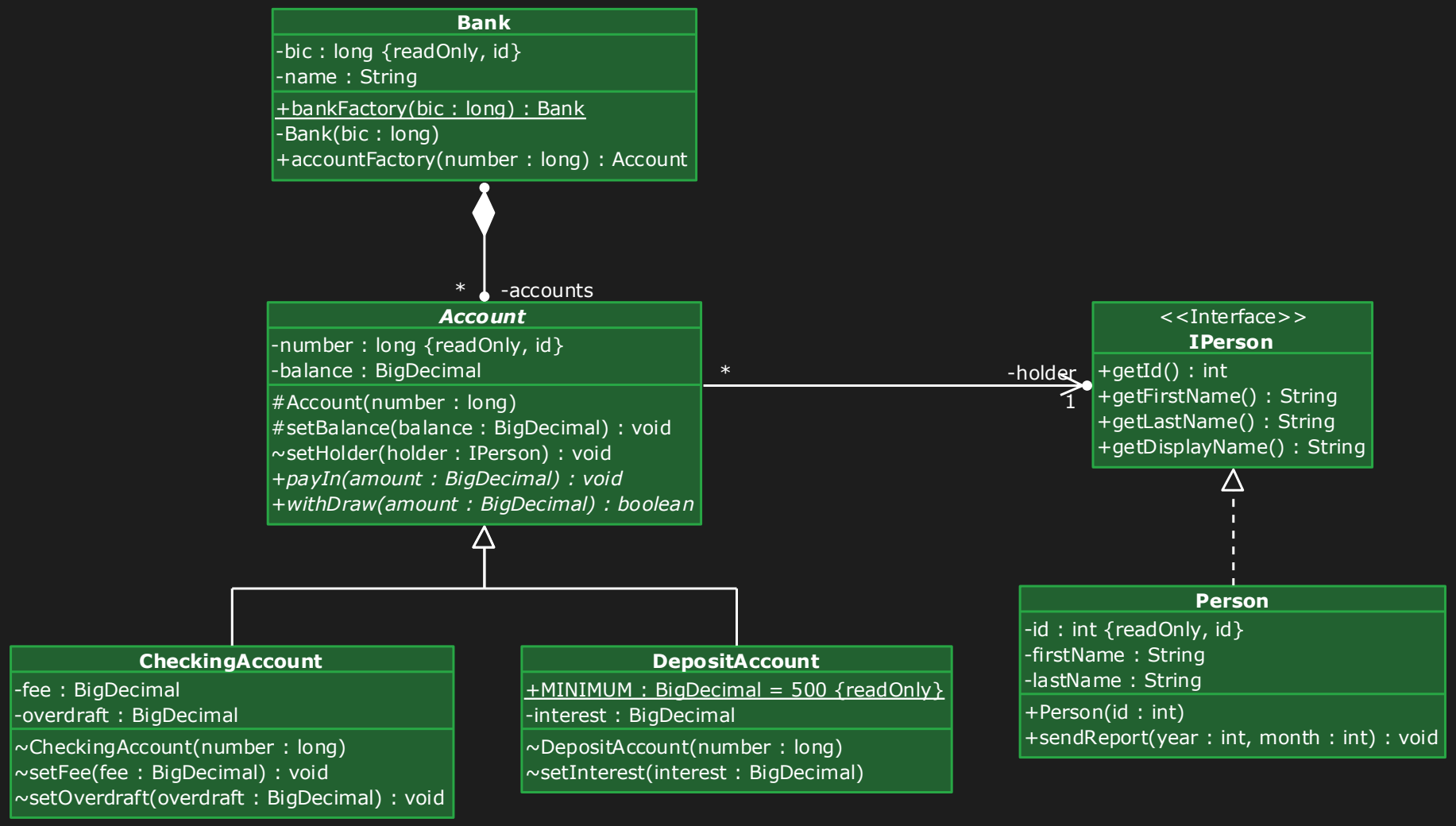
- Entspricht Generics in Java & C#, z.B. List<T>
- Es können mehrere Typparameter und Bedingungen für diese spezifiziert werden (hier nicht behandelt)
- Gebundene Klasse: Zuordnung von Typparameter T zu gewünschter Klasse über Bindungsabhängigkeit (Binding Dependency) mit «bind» oder als anonyme Bindung



Parametrisierte Klasse mit gebundenen Klassen

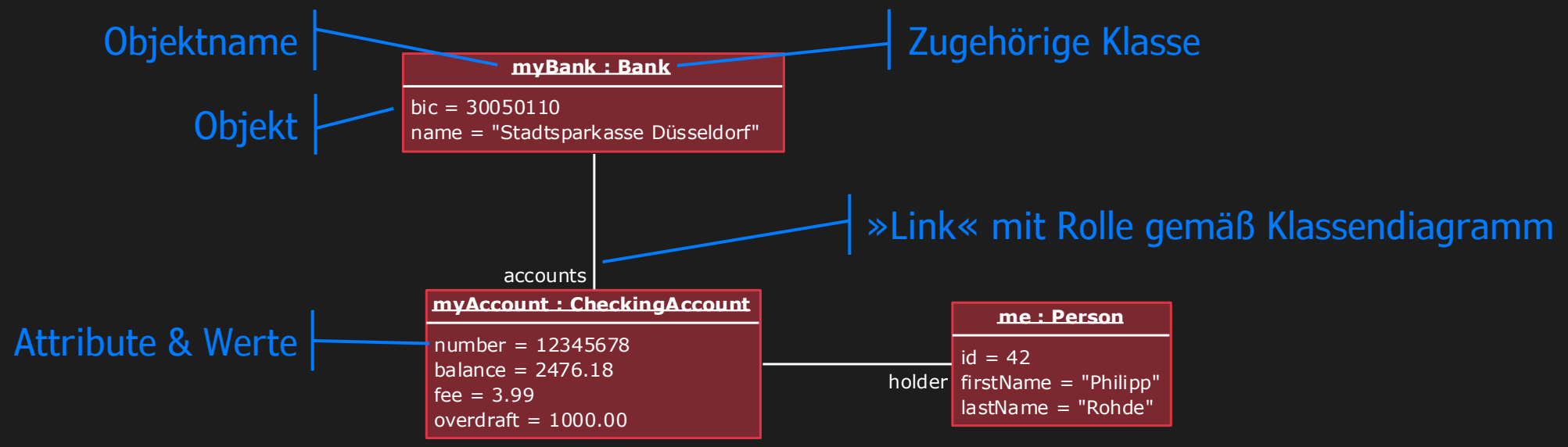
Anonym gebundene Klasse

Beispiel: Banking

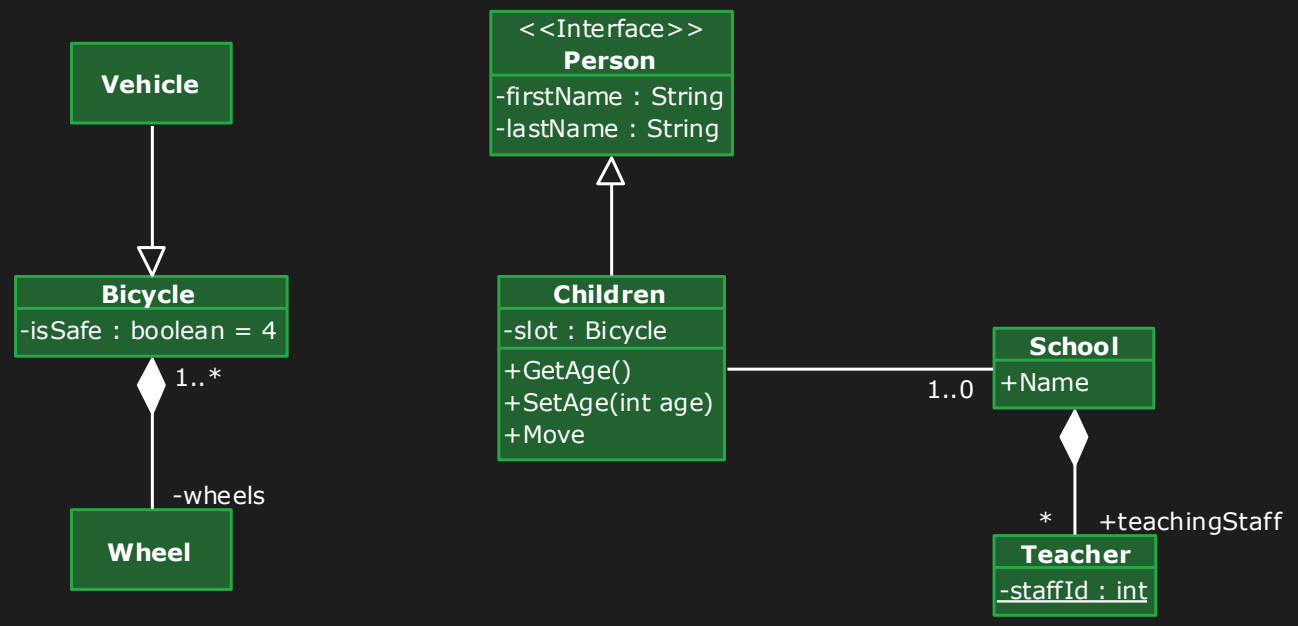


Objektdiagramm

- Zeigt Instanzen (Objekte) von Klassen & deren Beziehungen
- »Schnappschuss« der Attributwerte zur Laufzeit
- Darstellung bestimmter Konfigurationen, z.B. Start oder verbotener Zustand
- Beachte: ohne Angabe von Sichtbarkeiten
- Beispiel (zum vorherigen Klassendiagramm)



Klassendiagramm: 16 typische Fehler

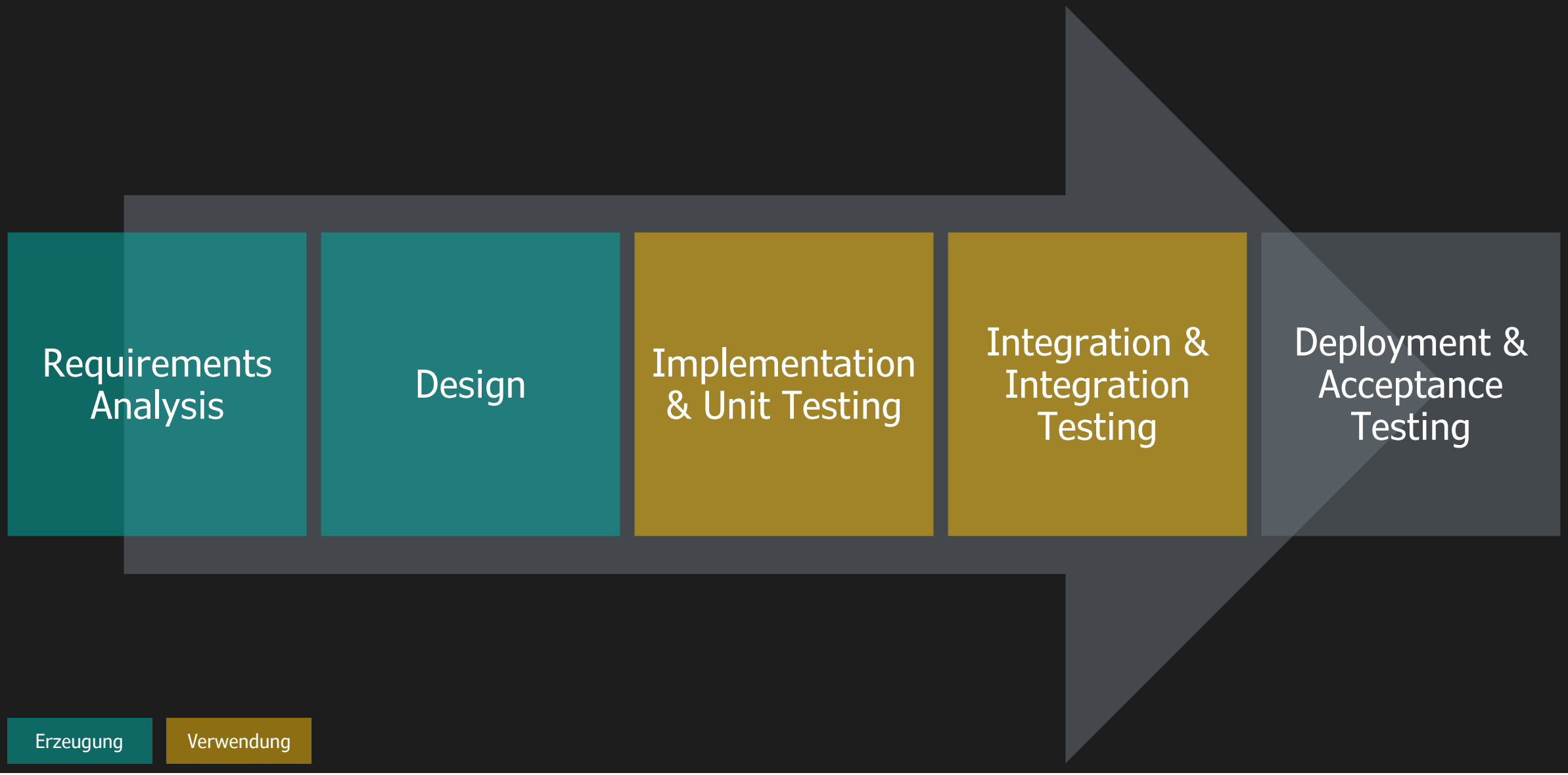


Abschnitt U.4

Aktivitätsdiagramme

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Relevante Phasen der Softwareentwicklung



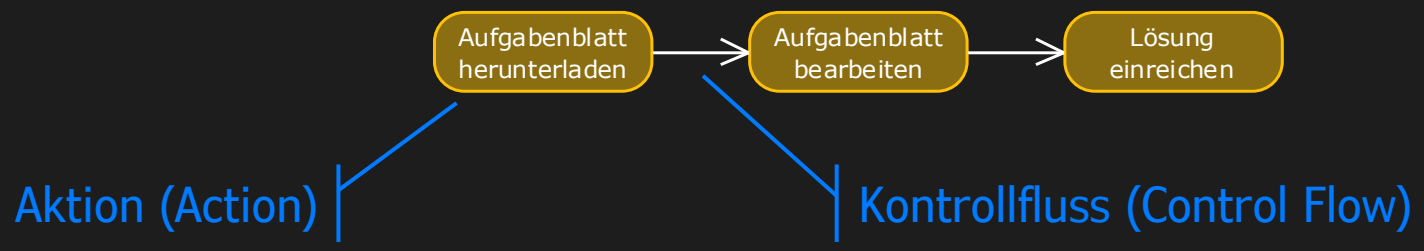
Motivation

- Modellierung von dynamischem Verhalten durch Kontroll- und Datenflüsse
- Requirements Analysis
  - Modellierung von Geschäftsprozessen oder zeitlich-logischer Reihenfolge von Tätigkeiten aus Sicht der Nutzenden einschließlich Alternativen etc.
  - Präzisierung der »inneren« Abläufe von Anwendungsfällen
- Design
  - Modellierung von Systemprozessen oder Abläufe zwischen Systemen
  - Präzise Modellierung von Algorithmen inklusive Nebenläufigkeit
- Implementation
  - Grundlage für die Implementierung von Klassen, Operationen und Abläufen von Operationsaufrufen
- Testing
  - Grundlage für Definition von Testfällen

Elemente

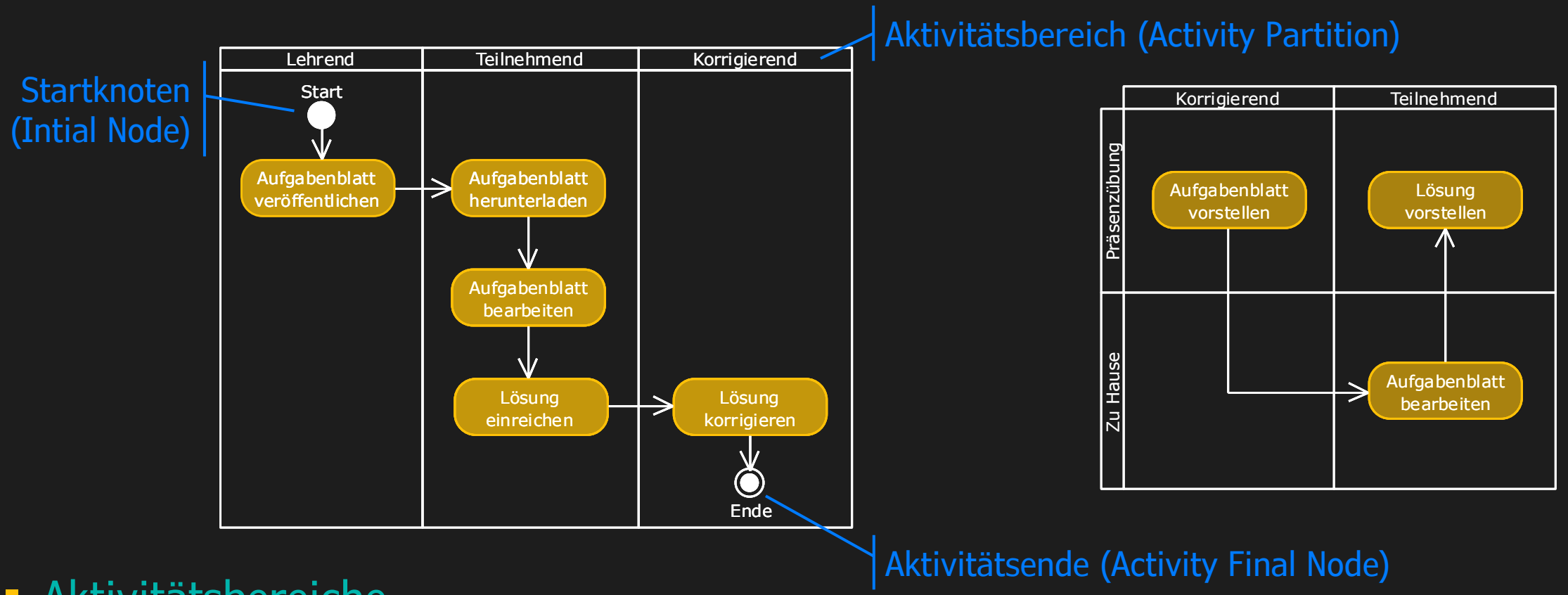
- Knoten
  - Ausführbaren Knoten (Executable Nodes), z.B. Aktionen, Aktivitäten
  - Objektknoten (Object Nodes), z.B. Dokumente, Klassen
  - Kontrollknoten (Control Nodes), z.B. Start, Ende, Verzweigungen
- Objekt- und Kontrollflüsse zwischen Knoten zur Modellierung der zeitlich-logischen Reihenfolge von Abläufen
- Parallele, verschachtelte und alternative Abläufe
- Aktivität- und Verantwortungsbereiche
- Ausnahmen und deren Behandlung
- Viele Gemeinsamkeiten mit Business Process Model and Notation (BPMN)

Aktionen & Kontrollfluss



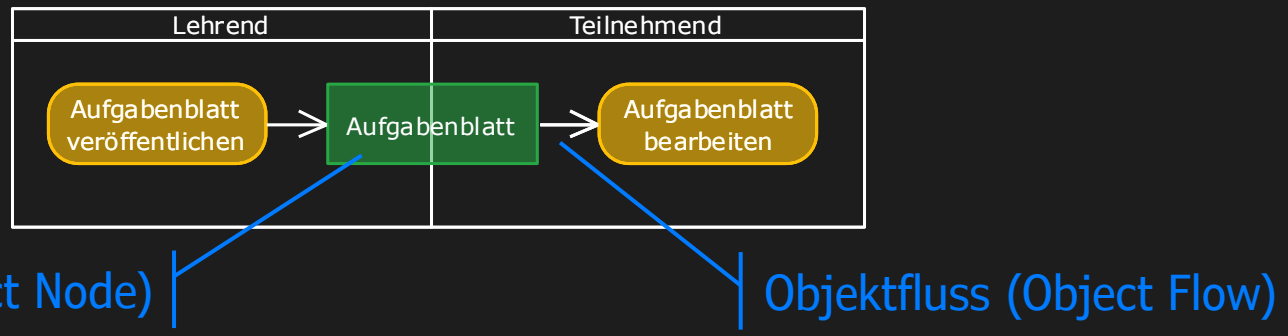
- Aktion: atomare, ausführbare Funktionalität
  - Atomar: für die Modellierung ist eine weitere Zerlegung nicht sinnvoll bzw. nicht relevant
  - Spezifikation von Vor-/Nachbedingungen für Ausführung ist möglich
  - Implementierung: typischerweise Operationsaufruf
- Kontrollfluss: gerichtete Verbindung zwischen Aktivitätsknoten
  - Ablauf bzw. Steuerung der Ausführung
  - Konnektoren sind möglich, falls Modellierung zu unübersichtlich wird
  - Implementierung: Einhaltung der Reihenfolge von Operationsaufrufen

Aktivitätsbereiche



- Aktivitätsbereiche
  - Gruppierung von Aktivitätsknoten zu Akteuren, Organisationseinheiten oder Systemen
  - Zuordnung von Aktivitätsknoten immer zu genau einem Bereich
  - Beachte: Objektknoten als Übergabe sind auf Grenzen möglich
  - Zweidimensionale Aufteilung möglich

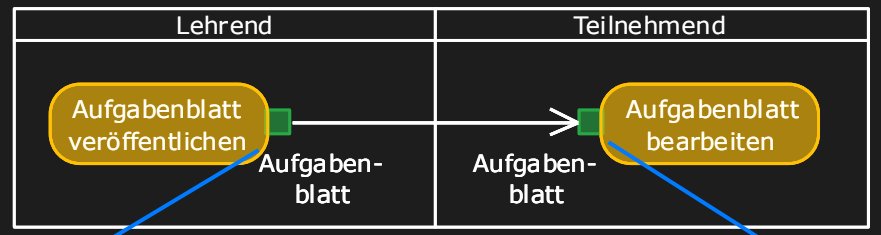
Objekte & Objektfluss



- Objektknoten: Übergabe von Objekten zwischen Aktivitätsknoten, z.B. Dokumente
  - Objekte werden in Aktivitätsknoten erzeugt oder verwendet
  - Objekte haben Zustände
  - Beachte: ein Objektknoten kann mehrere Objekte (als Klasseninstanzen) umfassen, z.B. als Queue (FIFO) oder Stack (LIFO)
  - Spezielle Knoten, z.B. Datenspeicher (Data Store Node)
  - UML-Notation wie bei Klassen- und Objektdiagrammen als »Objektname : Typ«
- Objektfluss: repräsentiert Transport von Objekten

Objekte: Pin-Notation & Objektzustände

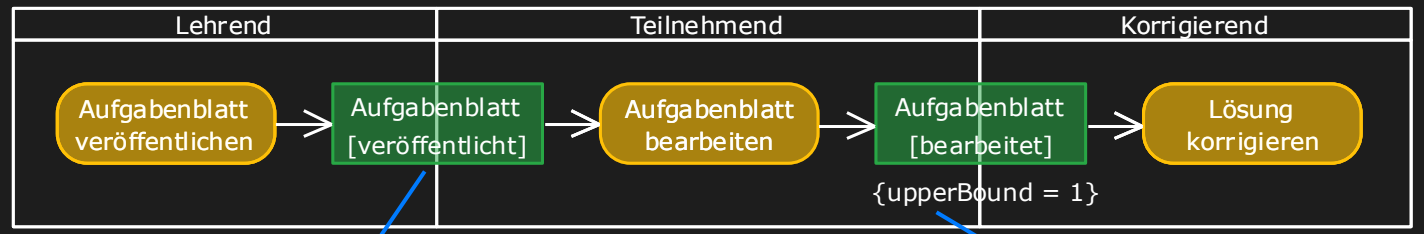
Pin-Notation:



Ausgang (Output Pin)

Eingang (Input Pin)

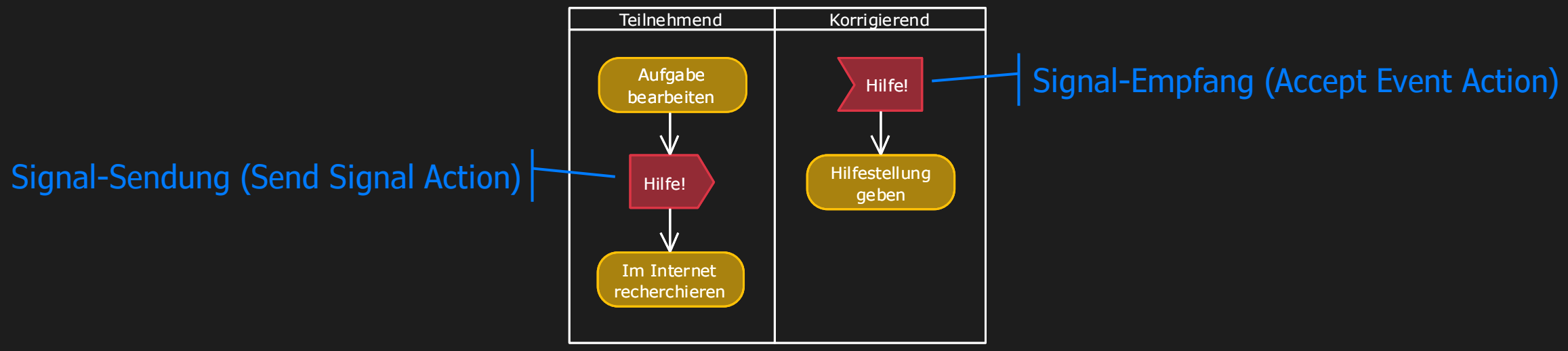
- Pin-Notation: Objekte als Ein- & Ausgabe von Aktivitätsknoten; Objekttypen müssen immer zusammenpassen
- Mehrere Pins können zu Parametersätzen zusammengefasst werden; Datenströme können spezifiziert werden



Objektzustand (Object State)

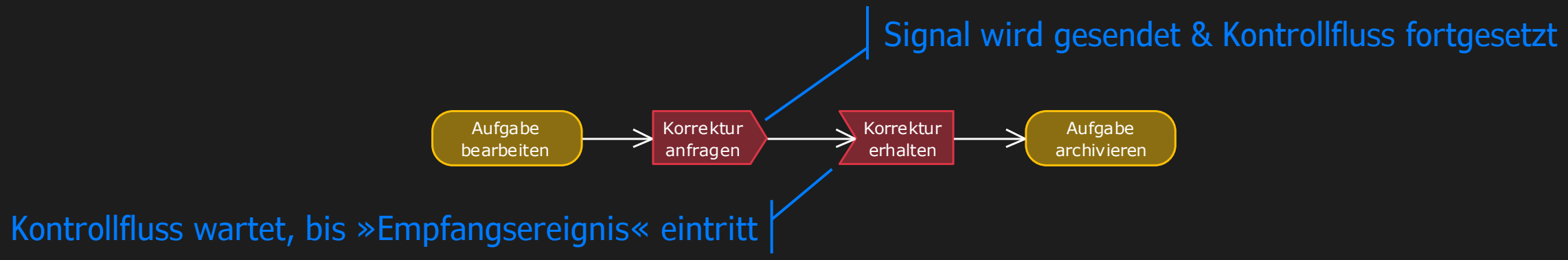
Spezifikation, z.B. Obergrenzen oder Ordnungsangaben (LIFO, FIFO)

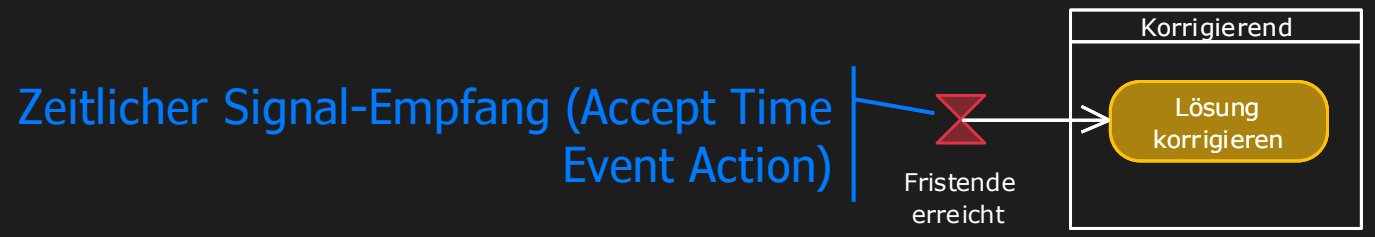
Signale



- Signal-Sendung: immer asynchron, der Kontrollfluss wird unmittelbar fortgesetzt
- Signal-Empfang
  - Nur, wenn umgebene Aktivität, Bereich bzw. Region aktiv ist
  - Ausgehender Kontrollfluss wird nach Empfang sofort ausgeführt
  - Ein Sender kann an mehrere Empfänger senden
  - Ein Empfänger kann von mehreren Sendern empfangen
- Implementierung: Observer (Java/C#) bzw. Events (C#)

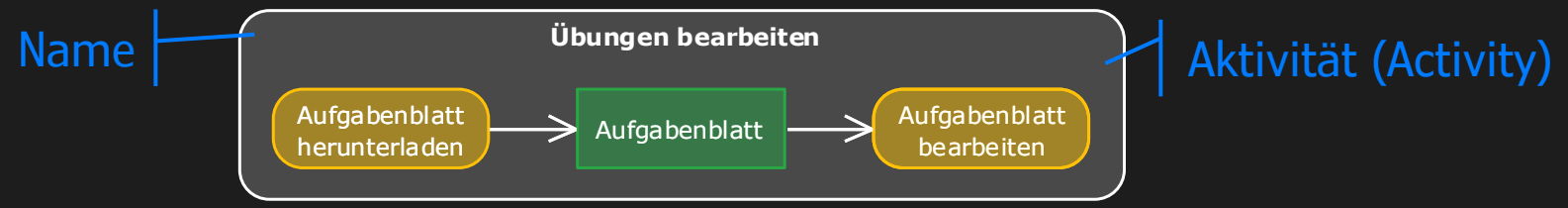
Kontrollfluss bei Signalen & Zeitsteuerung



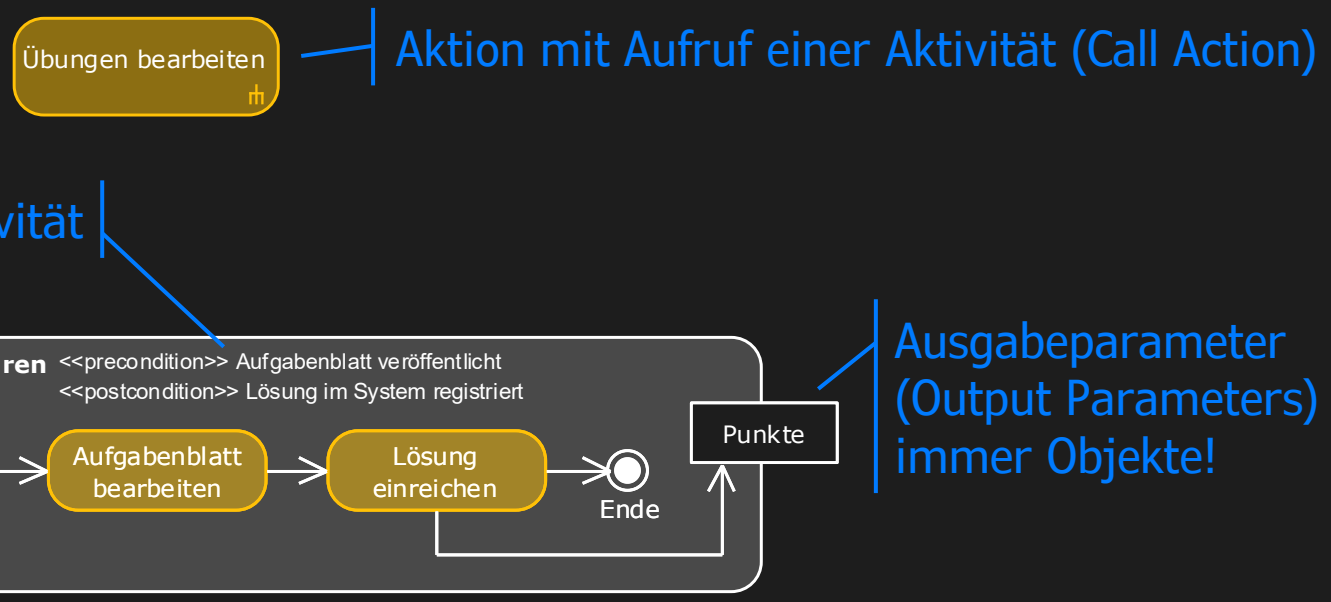


- Zeitangaben: Zeitpunkt (»um 22 Uhr«) oder Zeitraum (»nach 20 Minuten«)

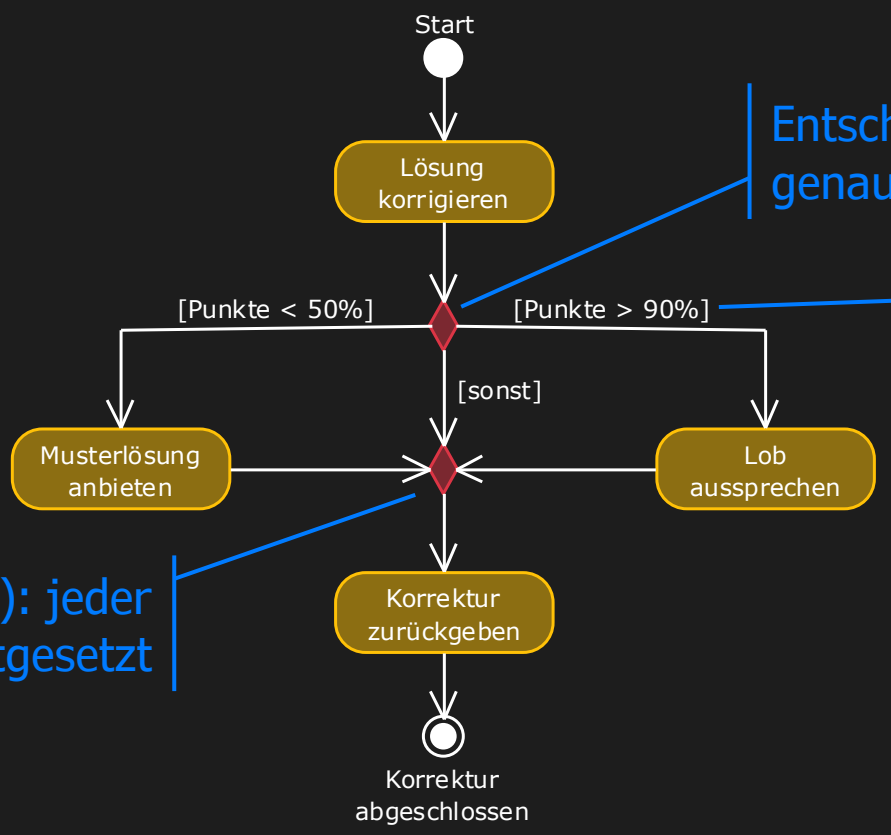
Aktivitäten



- Aktivität: geordnete Folge von Aktivitätsknoten, z.B. zur Wiederverwendung durch Aufruf aus anderen Diagrammen
- Gesamtes UML-Aktivitätsdiagramm wird auch als Aktivität interpretiert



Verzweigungen



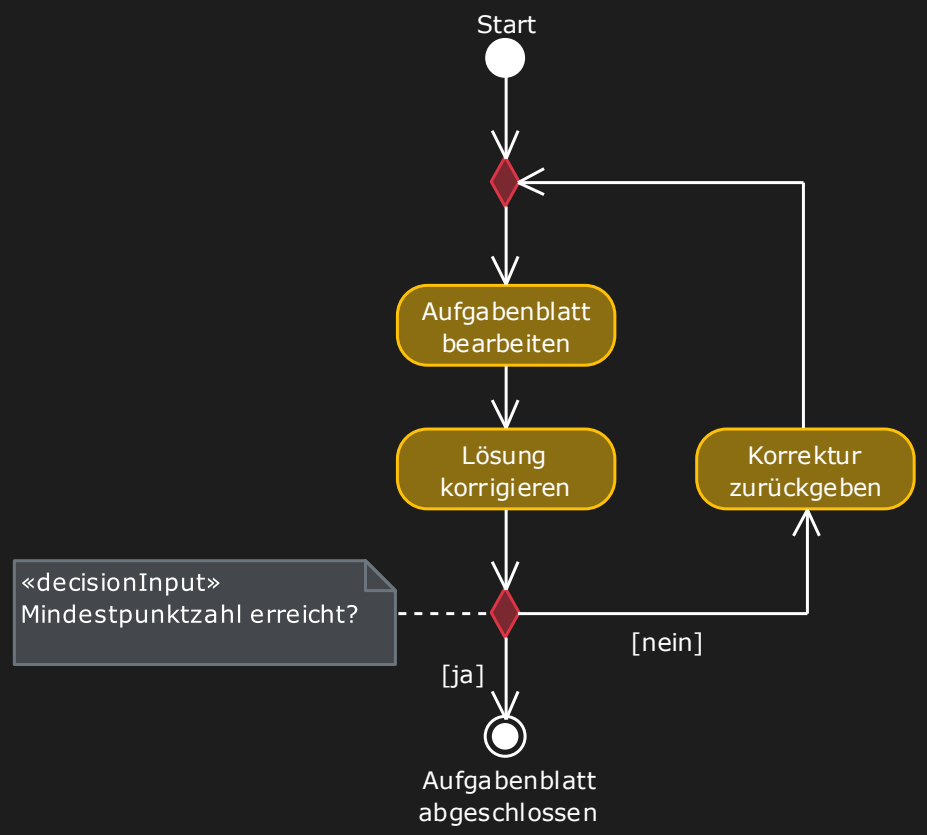
Entscheidungsknoten (Decision Node): genau eine Fortsetzung

Wächter (Guard): Boolesche Ausdrücke; müssen disjunkt & vollständig sein; Angabe in eckigen Klammern

Verbindungsknoten (Merge Node): jeder »eingehende Fluss« wird fortgesetzt

- Wächterbedingung: Prüfung beim Erreichen des Entscheidungsknotens
- Alternativ: Bedingung in Kommentar mit Schlüsselwort «decisionInput»
- Implementierung: If- oder Switch-Anweisungen; Merge als logisches Oder

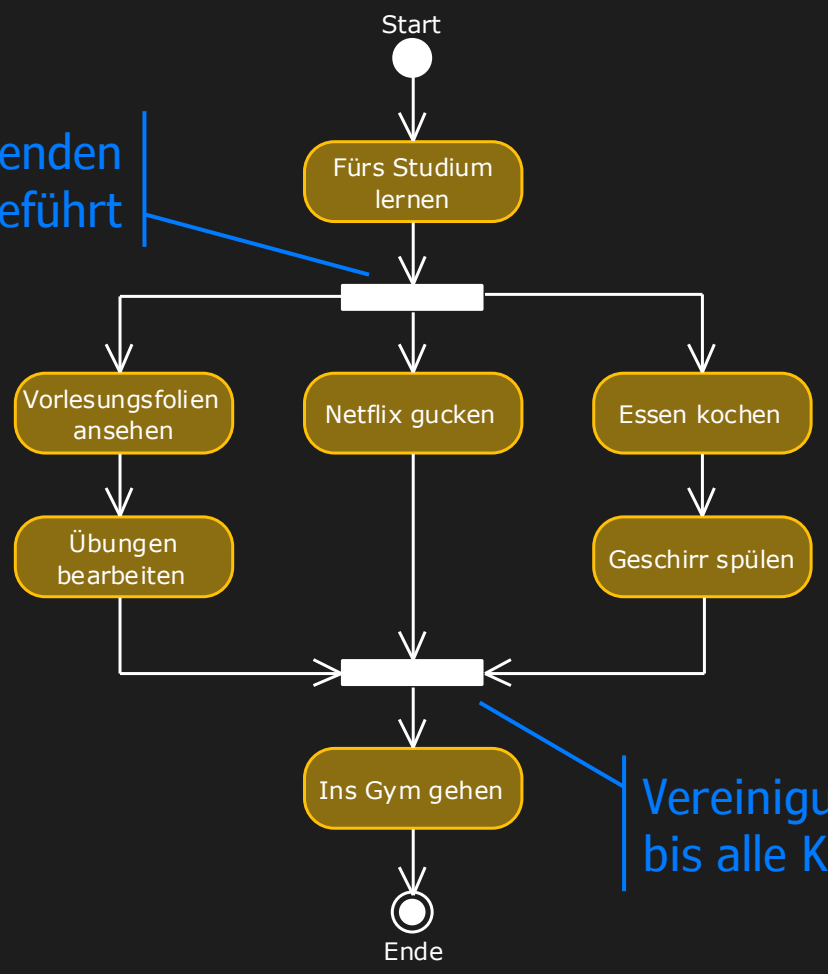
Verzweigungen: Schleifen



- Wächter beziehen sich auf das Ergebnis des «decisionInput»
- Implementierung: While-, Do-While-, For-, Foreach-Schleifen

Parallelisierung

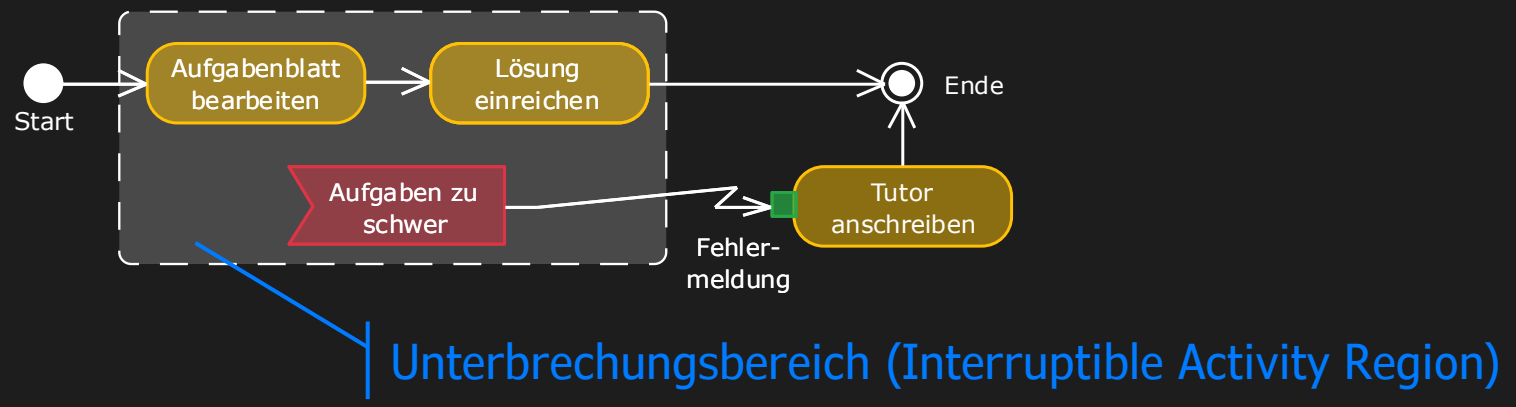
Gabelung (Fork Node): alle ausgehenden Kontrollflüsse werden ausgeführt



Vereinigung (Join): Synchronisation; wartet, bis alle Kontrollflüsse abgeschlossen sind

- Spezielle Vereinigungsspezifikationen sind möglich
- Implementierung: Threads bzw. Tasks; Join als logisches Und

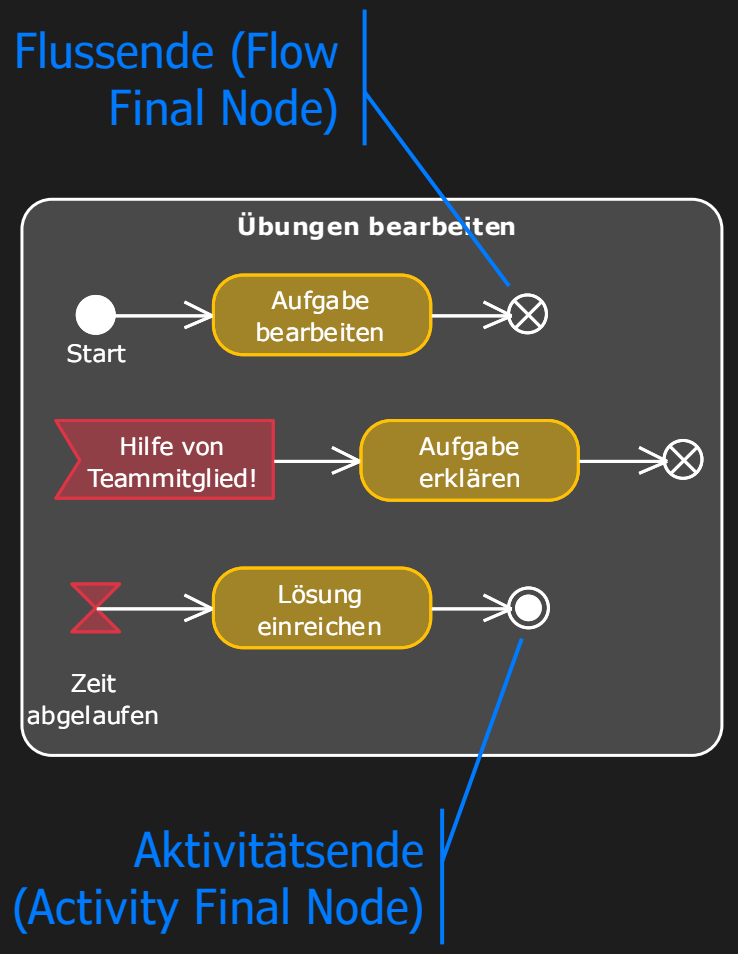
Ausnahmebehandlung



- Signal-Empfang: alle regulären (ggf. parallele) Abläufe des Unterbrechungs-bereichs werden abgebrochen & Bereich über Blitz-Kontrollfluss verlassen
- Implementierung: Throw Exception bzw. Catch Exception

Kontrollfluss: Start & Ende

- Start
  - Startknoten können Bedingungen / Ereignisse enthalten
  - Mehrere Startknoten sind erlaubt: jeder Startknoten löst eigenen Fluss aus
  - Startknoten können weggelassen werden, aber auslösende Ereignisse (z.B. Eingangsparameter, Signal-Eingang etc.) sollten eindeutig erkennbar sein
- Ende
  - Alle Kontrollflüsse (z.B. Verzweigungen, Parallelisierungen) sollten immer in Endknoten enden
  - Flussende: beendet nur eingehende Kontrollflüsse, aber keine parallelen Abläufe
  - Aktivitätsende: beendet alle Kontrollflüsse einer Aktivität und damit auch die Aktivität selbst

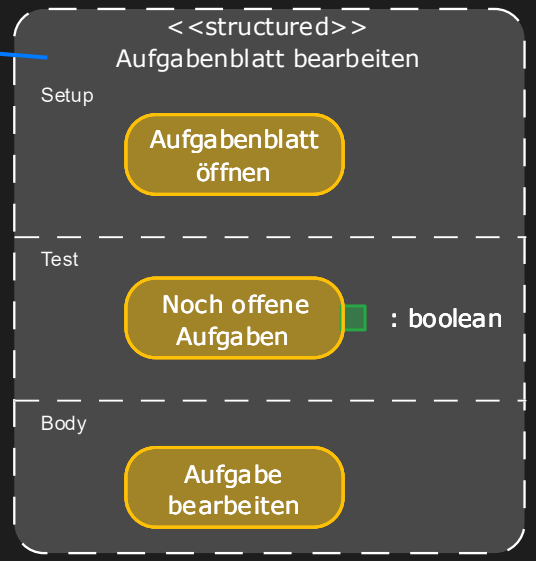


Kontrollfluss: Token

- Kontrollfluss kann durch Tokens simuliert werden
  - Jedes eintretende Startereignis erzeugt ein Token
  - Token folgenden dem Kontrollfluss
  - Ausführbare Knoten: Ausführung wird durch Eingang eines Tokens ausgelöst, erst nach »Abschluss« des Knotens wird der Token weitergegeben
  - Kontrollknoten: leiten Token weiter; Token werden ggf. vermehrt (z.B. Gabelung), verschmolzen (z.B. Vereinigung) & zerstört (z.B. Endknoten)
  - Token warten ggf. auf andere Tokens (z.B. Synchronisation bei Vereinigung)
  - Aktivitäten enden, wenn der letzte Token zerstört wurde
- Mehrere Kontrollflüsse bei ausführbaren Knoten
  - Mehrere ausgehende Kontrollflüsse: implizite Gabelung, d.h. nach Abschluss der Ausführung wird ein Token für jede Fortsetzung erzeugt
  - Mehrere eingehende Kontrollflüsse: implizite Vereinigung, d.h. es müssen erst alle Tokens ankommen, bevor die Ausführung des Knotens startet
- Beachte: der Objektfluss ist davon unabhängig, es können separate Objekt-Token verwendet werden

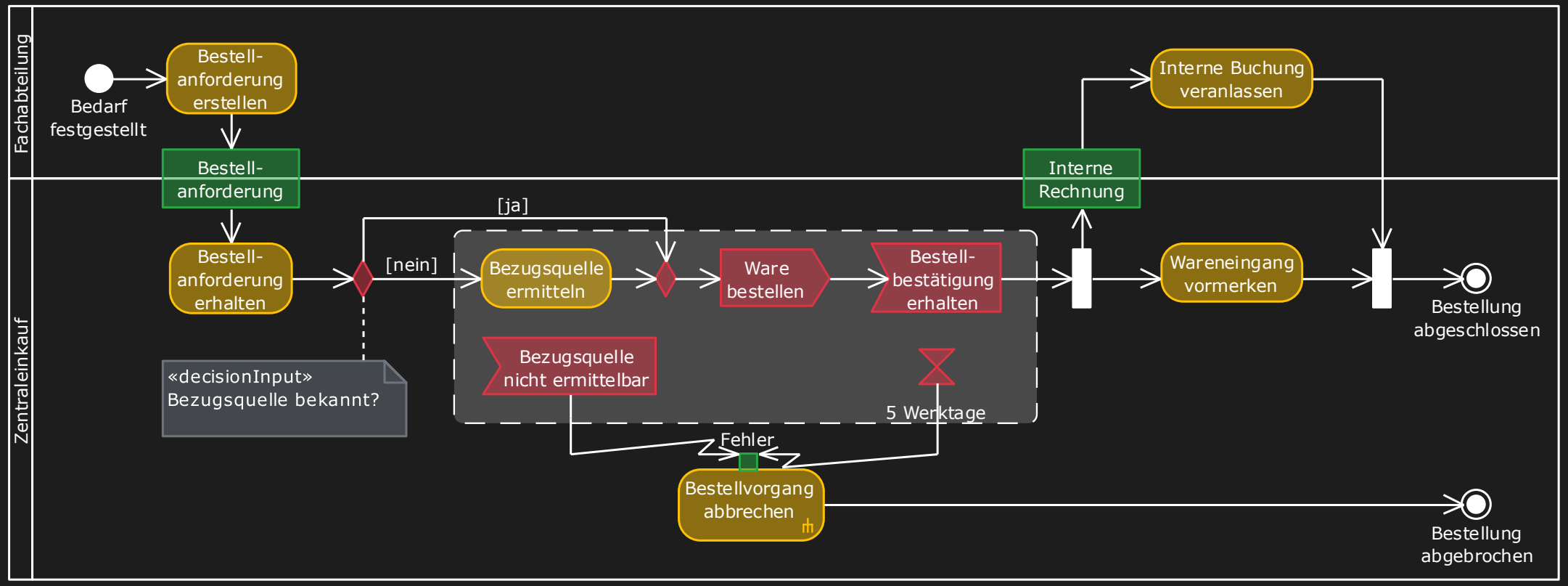
Weitere Elemente (ohne Details)

Strukturierte Aktivitäten (Structured Activity Node)

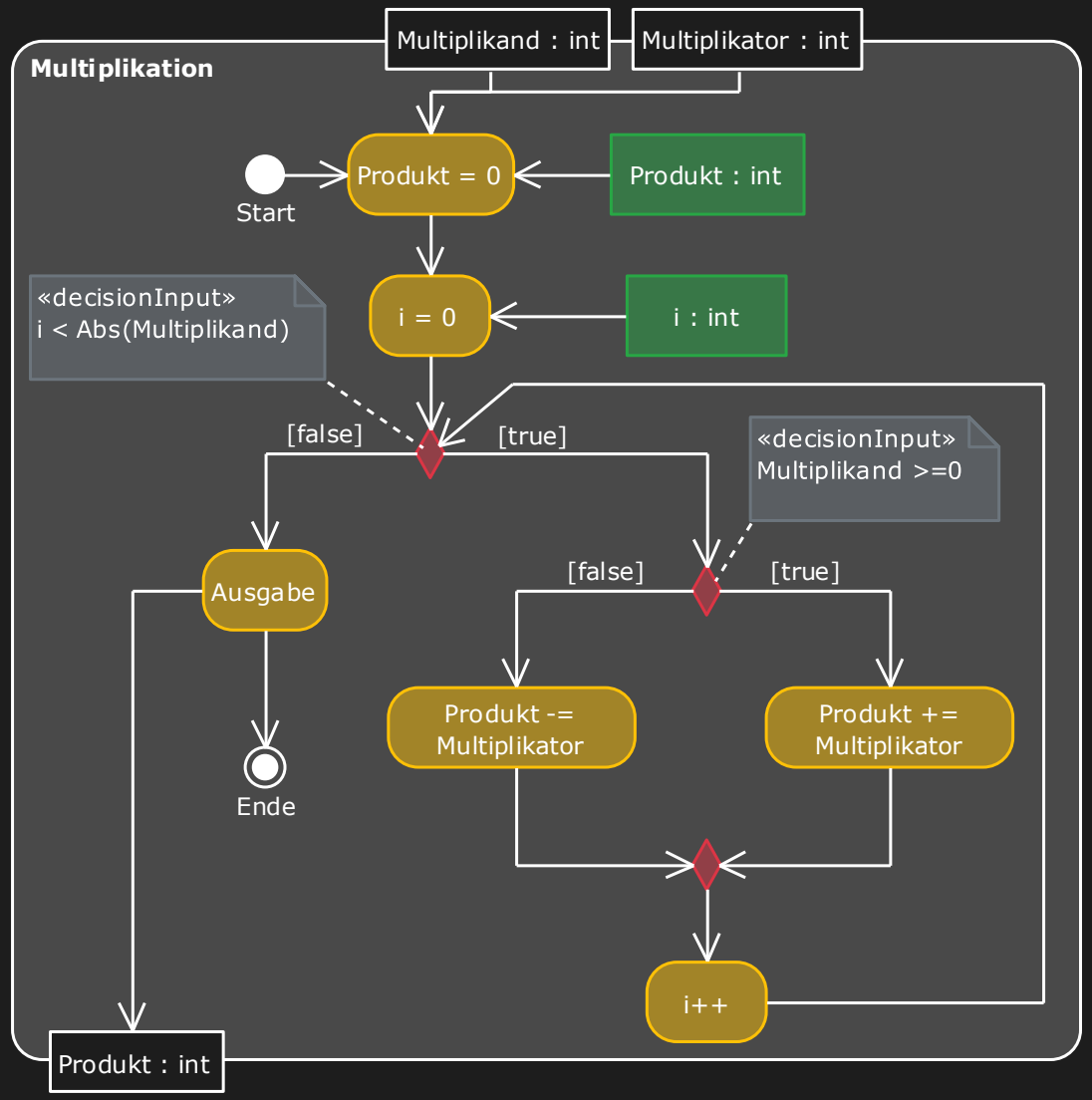


- Strukturierte Aktivitäten, z.B.
  - Schleifenknoten (Loop Node): Realisierung von Schleifen, siehe Beispiel
  - Bedingungsknoten (Conditional Node): Wahl von alternativen Abläufen
  - Sequenzknoten (Sequence Node): geordnete Ausführung in Reihenfolge
- Expansionsbereich: wiederholte Ausführung für mehrere Einzelelemente

Beispiel: Geschäftsprozess bei Anforderungsanalyse



Beispiel: Feinentwurf einer Klassenoperation

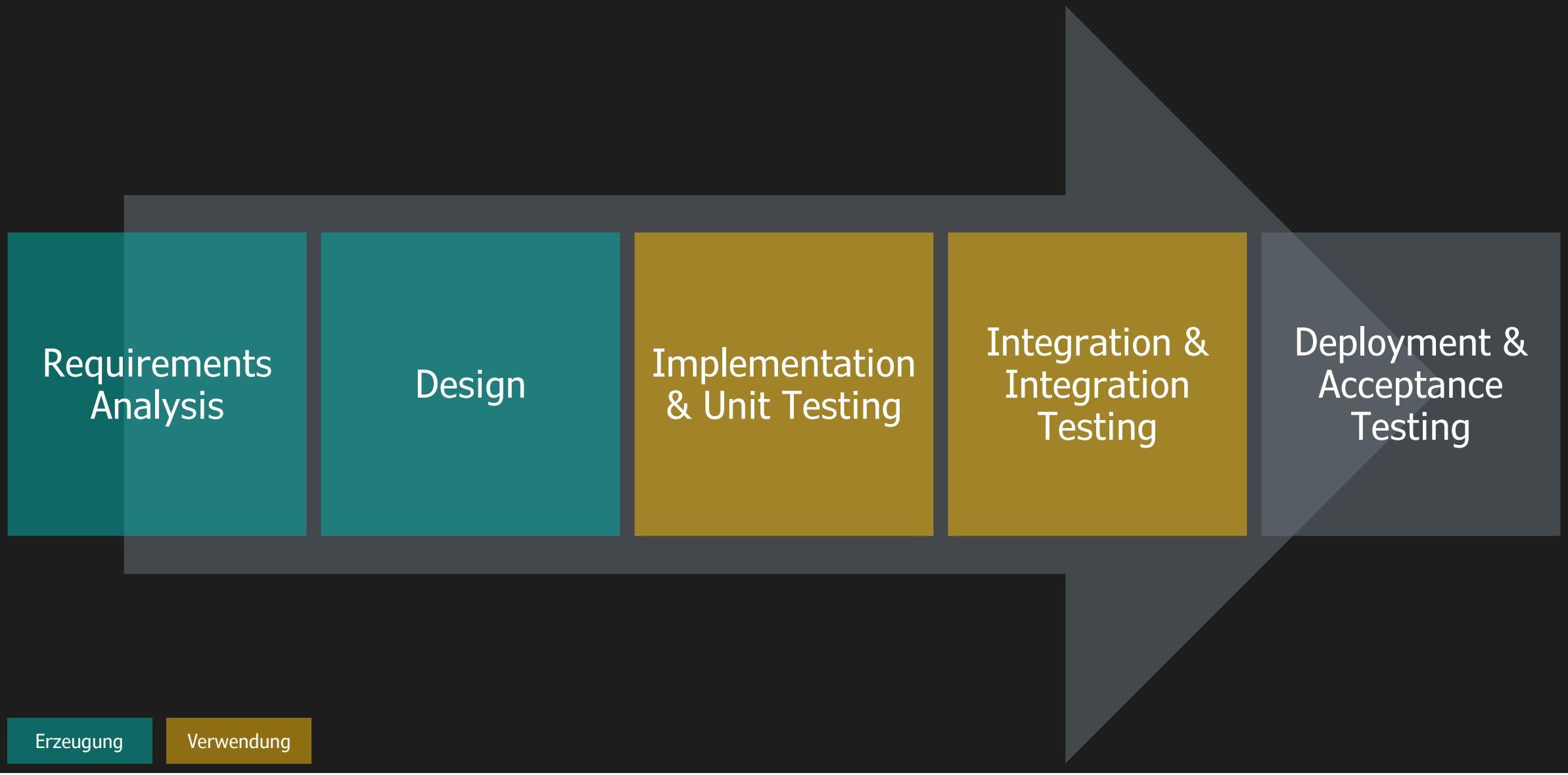


Abschnitt U.5

Zustandsdiagramme

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Relevante Phasen der Softwareentwicklung



Motivation

- Modellierung von dynamischem Verhalten durch Zustände von Objekten und mögliche Zustandsübergänge (Transitionen)
- Basieren auf deterministischen endlichen Automaten (DFA)
- Aktivitätsdiagramm
  - Fokus: mögliche Aktionen
  - Modellierung von Objekten & deren mögliche Zustände, aber oft implizit
- Zustandsdiagramm
  - Fokus: mögliche Reaktion des Systems
  - Modellierung der erlaubten Zustände & Zustandsübergänge eines Objekts
- Implementierung z.B. durch Entwurfsmuster »State« [→ Exkurs D]
- Testing: Zustandsübergangstest als Black-Box-Test [→ Kapitel 6]
- Verhaltens-Zustandsdiagramme, z.B. Modellierung einer Bedienoberfläche (GUI)
- Protokoll-Zustandsdiagramme, z.B. Modellierung von Kommunikationsprotokollen wie TCP/IP (hier nicht weiter behandelt)

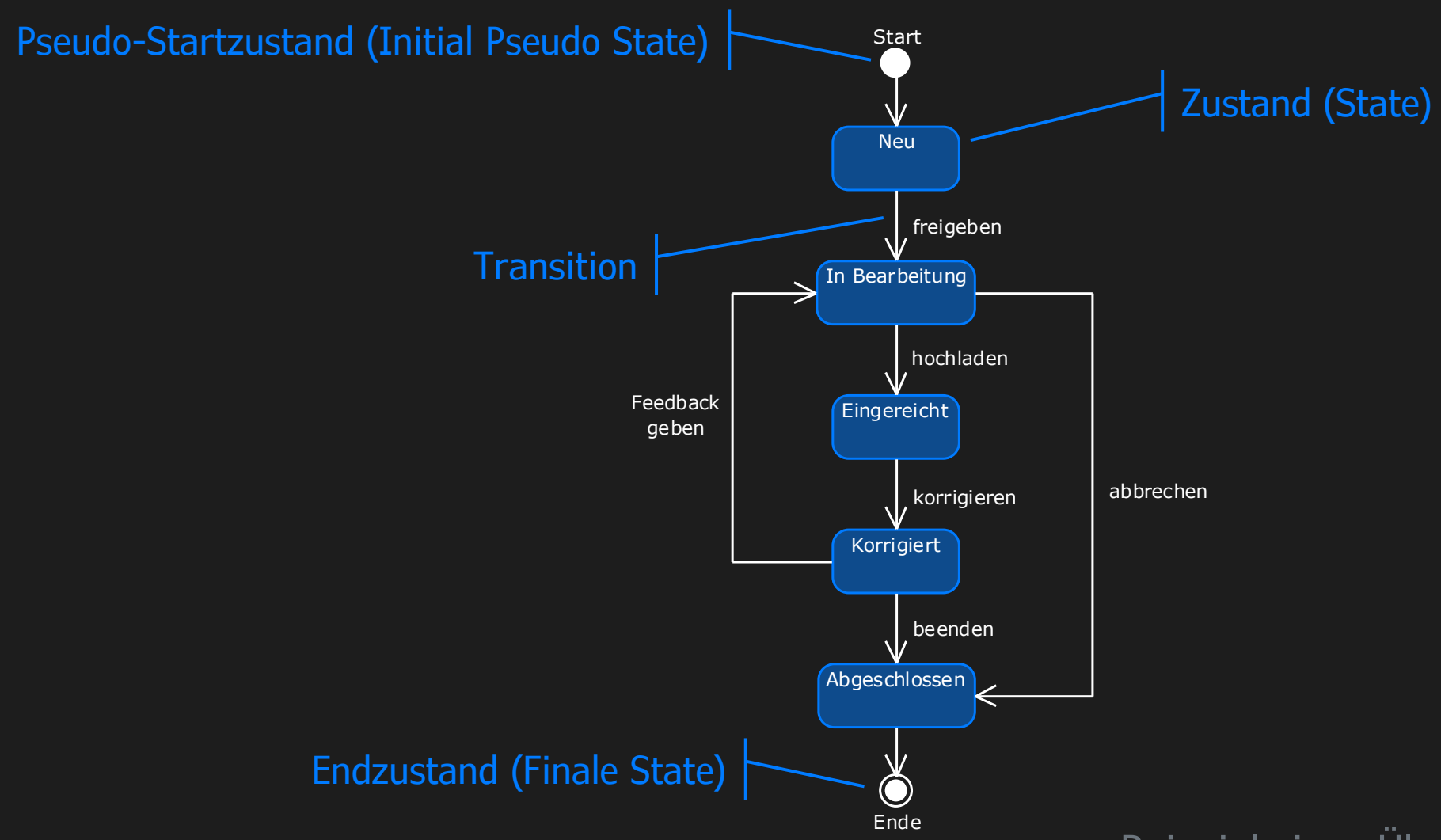
Zusammenhang mit Objektorientierung

- Lebenszyklus eines Objekts von Instanziierung bis Destruktion
- Objekte befinden sich jederzeit in genau einem Zustand
  - Zustand ist gegeben durch die Werte der Attribute
  - Beispiel: Person mit Namen, Geburtstag, aktuelle Position etc.
- Objekte haben ein Verhalten
  - Gegeben durch die Operationen der Klasse
  - Verhalten verändert im Allgemeinen den Zustand des Objekts
  - Beispiel: Person kann heiraten, Unternehmen verlassen, spazieren gehen etc.

Zustände & Transitionen

- Zustand
  - Modelliert eine Situation, in der definierte Bedingungen gelten
  - Ein Objekt befindet sich immer in genau einem aktiven Zustand
- Transition
  - Gerichtete Beziehung zwischen zwei Zuständen: möglicher Zustandsübergang zwischen Quell- und Zielzustand
- Pseudo-Startzustand
  - Zeigt den Start der Ausführung an
  - Genau ein Startzustand (pro Ebene bzw. Region); dieser darf keine eingehende Transition haben
- Endzustand
  - Zeigt das Ende der Ausführung eines Bereichs an
  - Mindestens ein Endzustand (pro Ebene bzw. Region), mehrere Endzustände sind erlaubt; diese dürfen keine ausgehenden Transitionen haben
- Terminator
  - Die Ausführung des gesamten Diagramms wird abgebrochen

Beispiel: Zustände & Transitionen

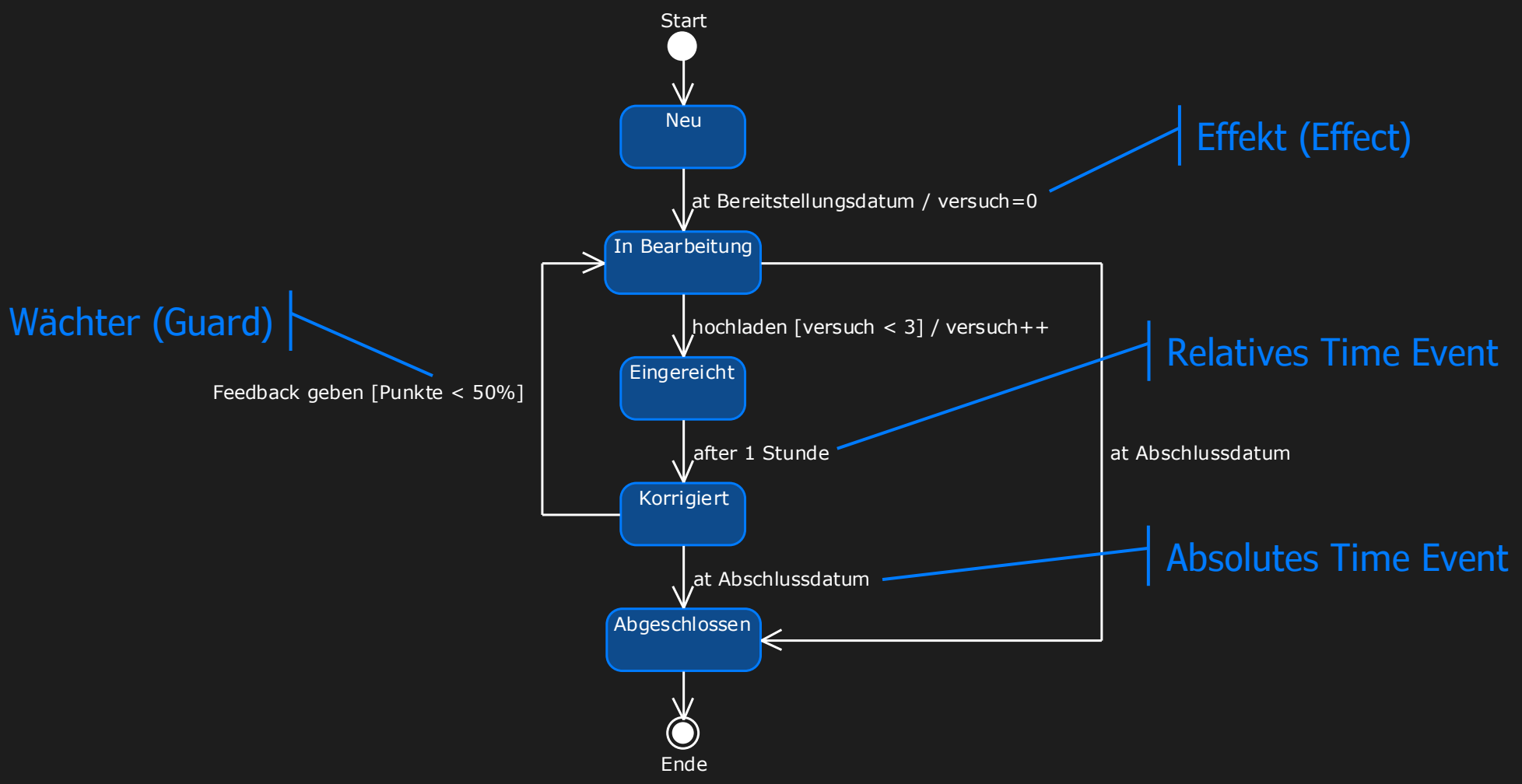


Beispiel eines Übungsbetriebs

Ereignisse, Wächter & Effekte

- Transitionen werden durch Ereignisse (Events) ausgelöst
  - Ereignisse können nur für Objekt im aktiven Zustand eintreten
  - Objekt kann diese ignorieren oder reagieren, indem Aktion ausgeführt & Zustand geändert wird
- Angabe an Transition: Ereignis [Wächter] / Effekt
- Ereignistypen als Auslöser einer Transition
  - Call Event: eine Operation wird aufgerufen
  - Signal Event: Empfang eines (asynchronen) Signals
  - Change Event: boolesche Bedingung für Attributwerte ist erfüllt (when)
  - Time Event: Zeitpunkt (at) oder nach einer Zeitspanne (after)
  - Any Receive Event: alle anderen Auslöser (all)
- Wächter
  - Transition wird nur ausgeführt, wenn boolesche Bedingung wahr ist
- Effekt
  - Aktion, die bei Transition zusätzlich ausgeführt wird, z.B. Variablenwert setzen

Beispiel: Ereignisse & Effekte



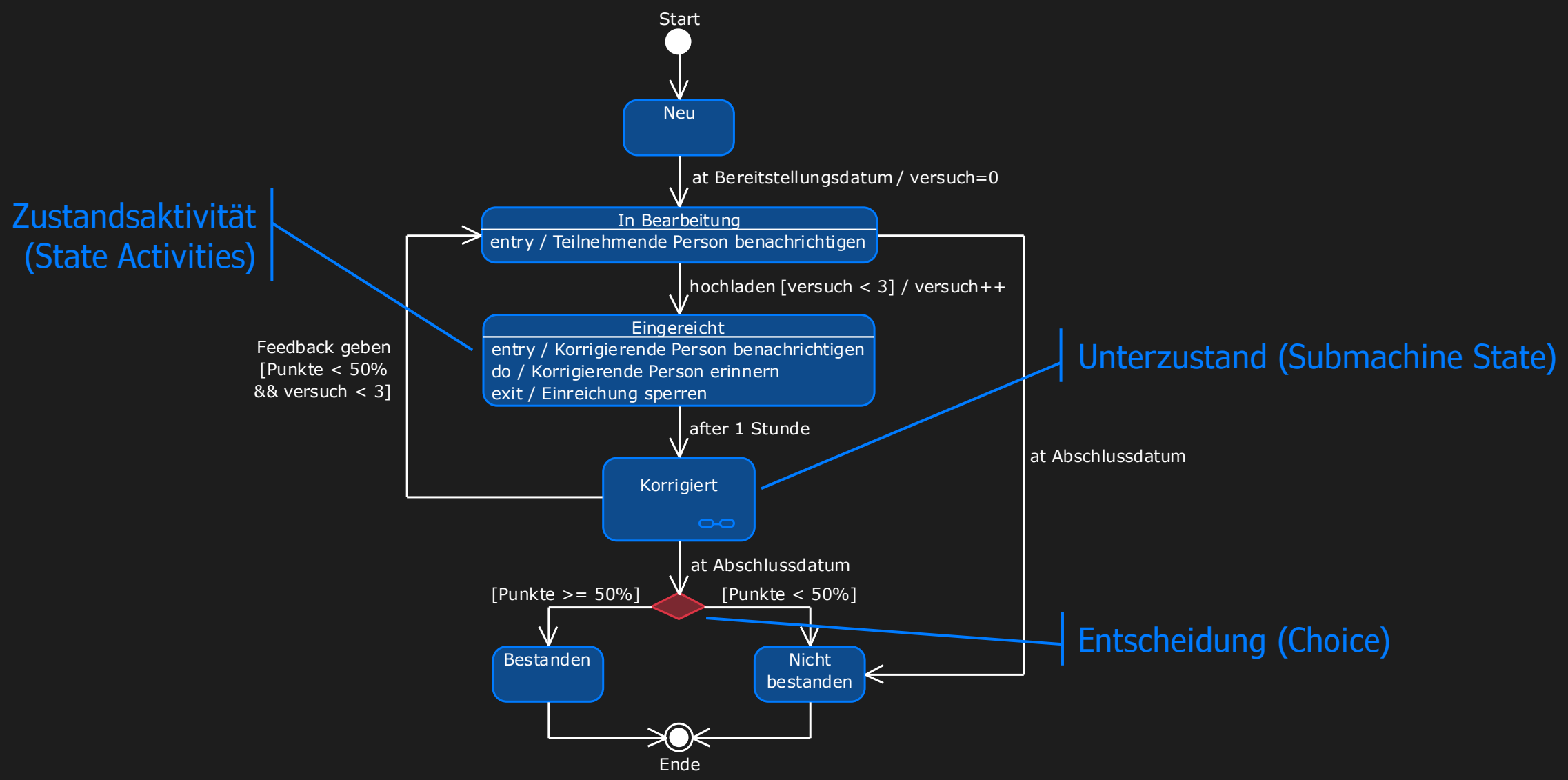
Ereignisse: Regeln

- Keine Deadlocks
  - In normalen Zuständen muss immer eine geeignete Transition zur Fortsetzung möglich sein
- Pseudo-Startzustände
  - Diese können immer verlassen werden: Bedingungen (Ereignis bzw. Wächter) an ausgehender Transition sind nicht erlaubt
- Wächter-Bedingungen
  - Disjunkt & vollständig: es muss immer eindeutige Transition möglich sein
- Schleifen
  - Transition eines Zustands zu sich selbst ist erlaubt, aber immer mit Ereignis und/oder Wächter, da sonst Endlosschleife
- Mehrere Ereignisse, ggf. mit Wächter & Effekt: durch Komma getrennt

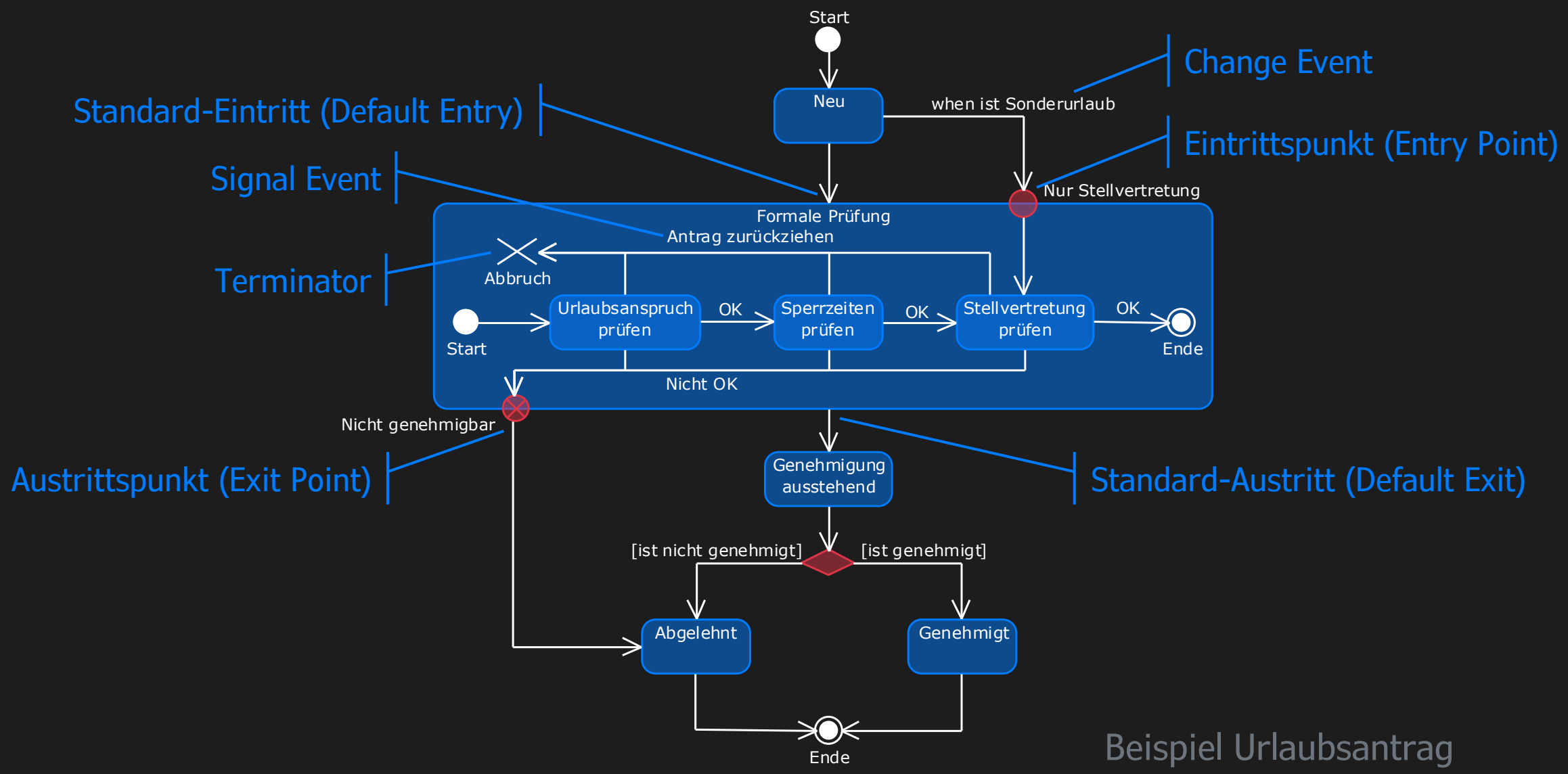
Weitere Elemente

- Zustandsaktivität
  - Entry: Aktion wird beim Betreten des Zustands einmal ausgeführt und zu Ende gebracht, bevor weitere Aktion stattfinden kann
  - Do: Aktion startet nach Betreten / nach Entry-Aktion; wird so lange ausgeführt, bis sie endet oder Zustand verlassen wird
  - Exit: Aktion wird vor dem Verlassen des Zustands einmal ausgeführt
- Entscheidung: dynamische Verzweigung, Auswertung der Wächter bei Erreichen
- Kreuzung: statische Verzweigung, Auswertung der Wächter bereits beim Start
- Unterzustand: Verweis auf separates Diagramm; Hierarchien sind möglich
- Zusammengesetzter Zustand
  - Hierarchie von Zuständen: explizit im gleichen Diagramm oder Verweis auf separates Diagramm
  - Logik für Ein- & Austritt: Standard-Eintritt/-Austritt bzw. Eintritts- oder Austrittspunkte
  - Beachte: Ein-/Austrittspunkte nur bei zusammengesetzten Zuständen
  - Weitere Möglichkeiten (hier nicht behandelt): expliziter Eintritt / Austritt; Eintritt über flache / tiefe Historie; Austritt über Endzustand / Terminator; ausgehende Transition

Beispiel: Weitere Elemente



Beispiel: Zusammengesetzte Zustände

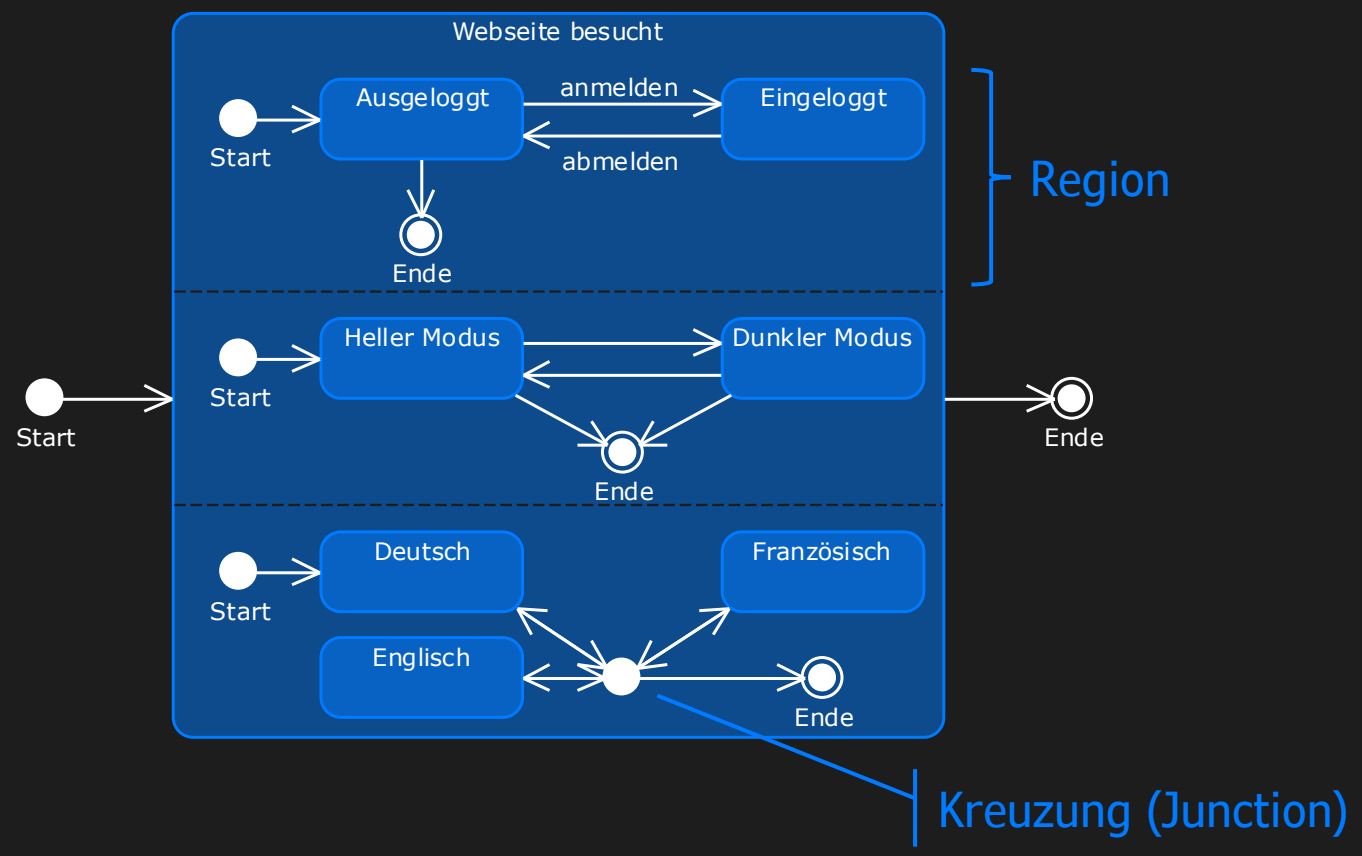


Beispiel Urlaubsantrag

Regionen

- Region
  - Aufteilung von zusammengesetzten Zuständen in disjunkte Bestandteile
  - Objekt kann gleichzeitig & unabhängig voneinander bestimmte Zustände annehmen
  - Transitionen jeder Region sind nebenläufig
- Jede Region kann eigene Start- und mehrere Endzustände besitzen
  - Beim Betreten des zusammengesetzten Zustands werden die Startzustände aller Regionen aktiv
  - Der zusammengesetzte Zustand kann nur verlassen werden, wenn alle enthaltenen Regionen einen Endzustand erreicht haben
  - Ein Terminator führt dazu, dass der zusammengesetzte Zustand sofort verlassen wird
- Transitionen zwischen Regionen sind nicht erlaubt
- Weitere Konstruktionen: Generalisierung & Spezialisierung

Beispiel: Regionen



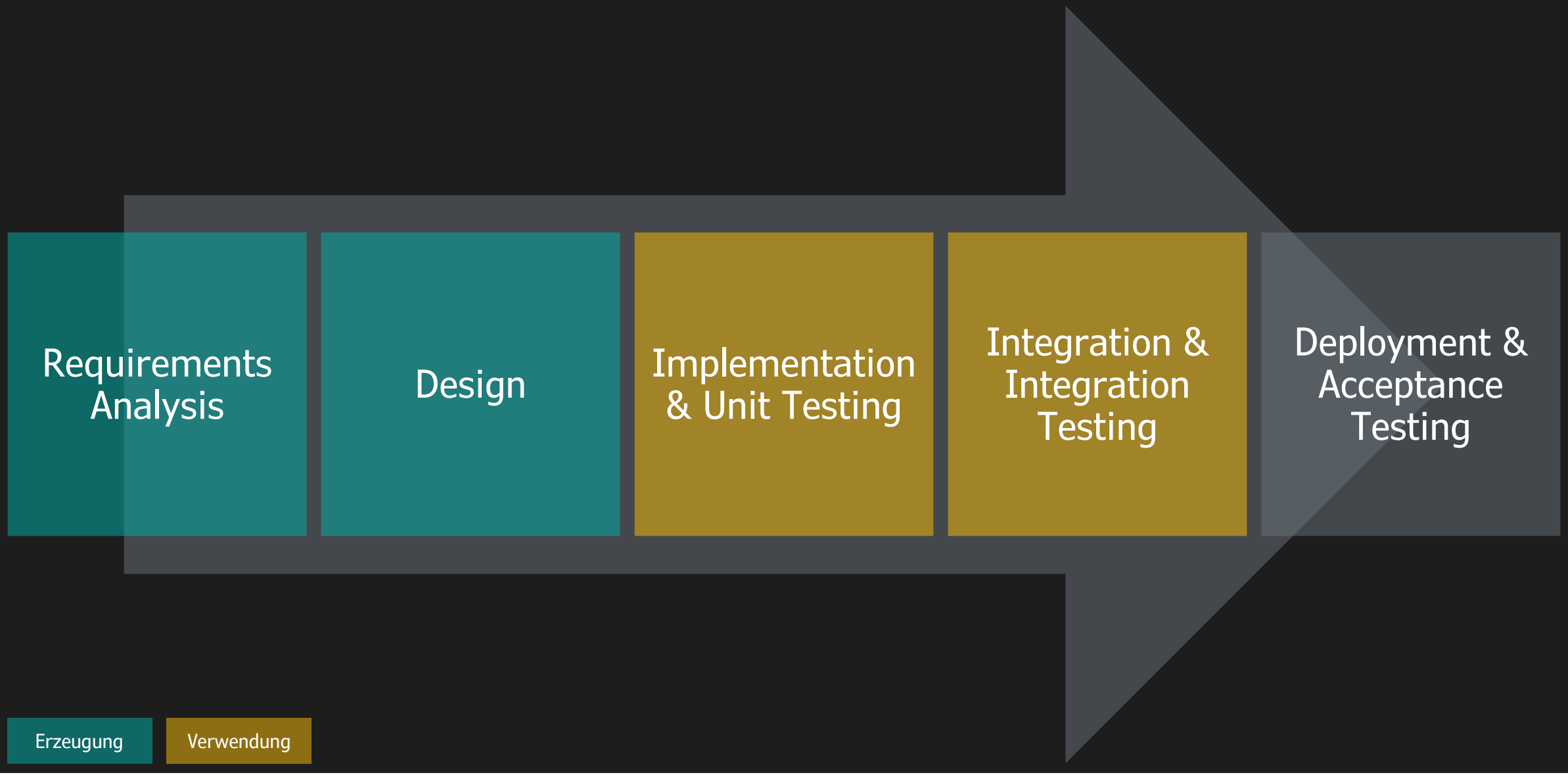
Beispiel Webseite

Abschnitt U.6

Sequenzdiagramme

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

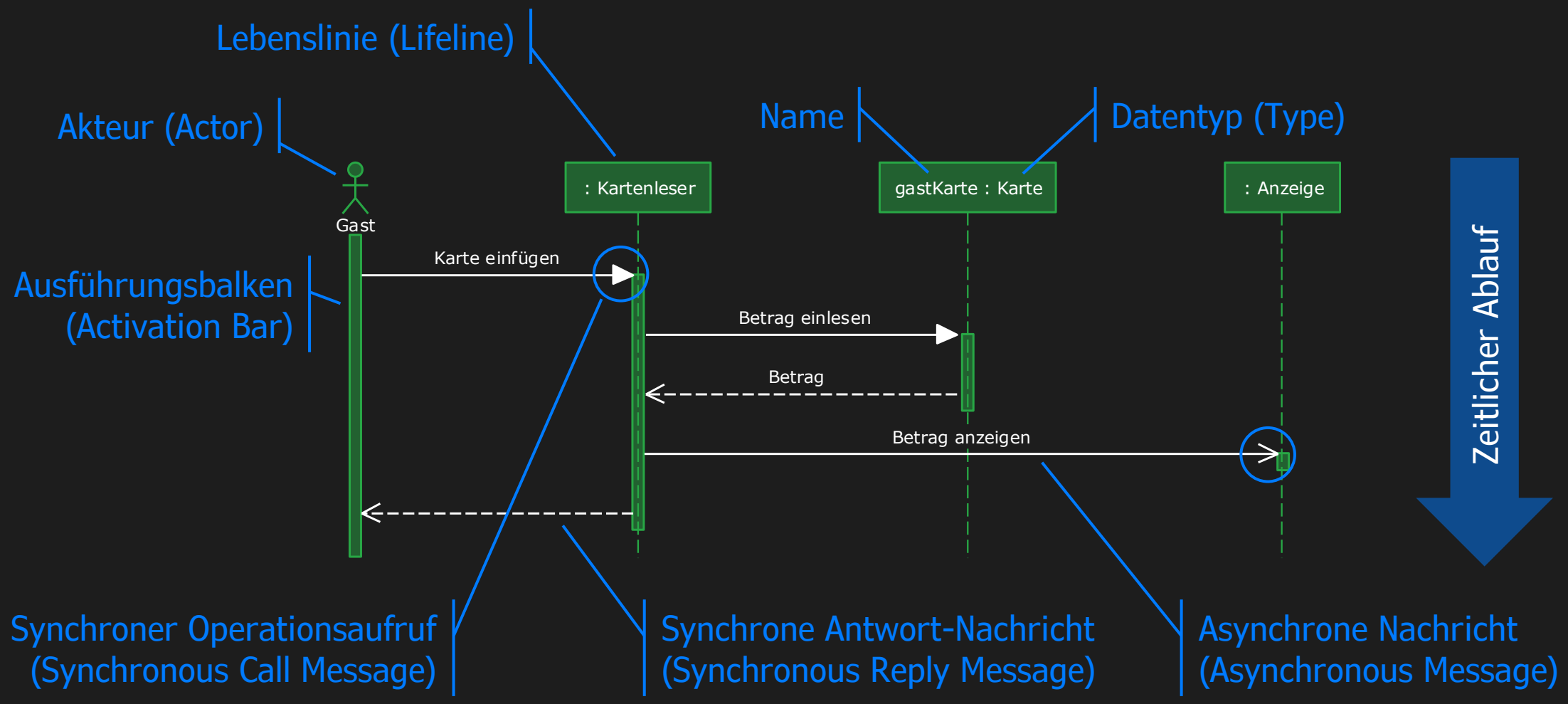
Relevante Phasen der Softwareentwicklung



Motivation

- Modellierung von dynamischem Verhalten
  - Interaktionen zwischen Objekten
  - Nachrichtenfluss, d.h. zeitlicher Ablauf und Reihenfolge von Nachrichten zwischen den Objekten während Interaktionen
  - Beachte: Aufruf einer Klassenoperation ist formal auch »Nachricht«
- Requirements Analysis
  - Nachrichtenflüsse in Geschäftsprozessen, insbesondere zwischen Akteuren und Systemen
  - Verfeinerung von Anwendungsfällen
- Design
  - Interaktion von Nutzenden mit GUI, z.B. Ein- & Ausgabe von Daten
  - Modellierung der technischen Interaktion zwischen Komponenten und Klassen, insbesondere Operationsaufrufe & Rückgaben
- Implementation & Testing
  - Fachliche Spezifikation für die Implementierung
  - Ableitung von Testfällen

Beispiel: Kartenlesegerät (vollständig)

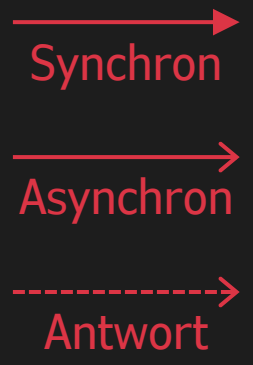


Lebenslinien

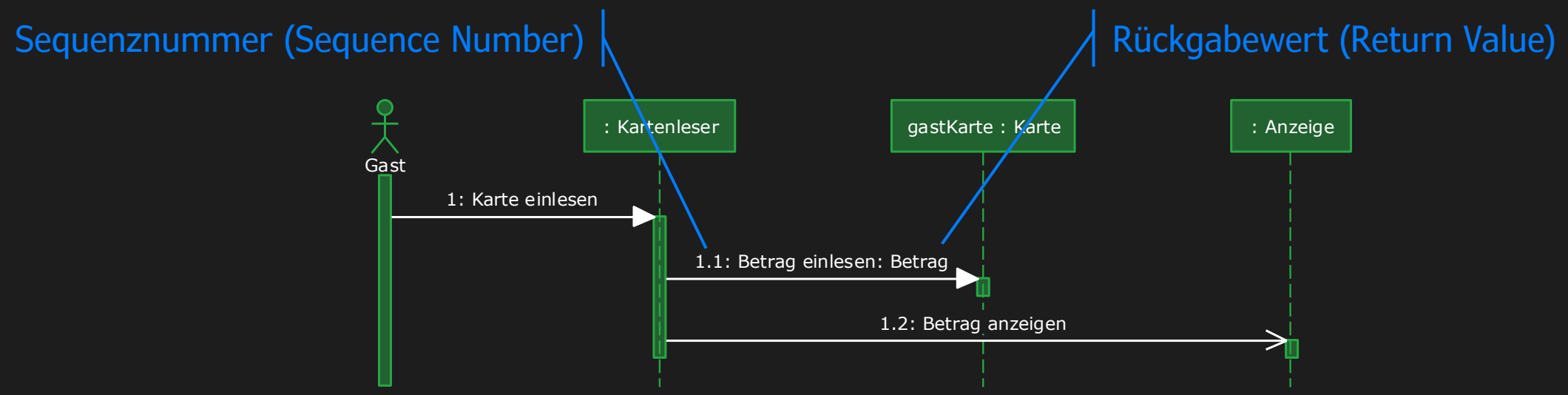
- Lebenslinie
  - Repräsentiert genau einen Teilnehmer einer Interaktion
  - Entweder Akteur oder Komponente / Klasse
  - Angaben: Name optional, Typ verpflichtend
- Anordnung
  - Horizontal: Lebenslinien; Reihenfolge egal, aber Initiator der Interaktion im Allgemeinen links
  - Vertikal: zeitliche Reihenfolge der Nachrichten (von oben nach unten)
- Ausführungsbalken
  - Zeigt die aktive Phase während einer Interaktion
  - Passive Phasen durch gestrichelte Linie
  - Ausführungsbalken können sich auch überlagern, z.B. bei parallelen Aktivitäten
- Explizites Ende einer Lebenslinie nach Destruktionsnachricht mit Stopp-Symbol (Achtung: nie bei Personen)

Nachrichten

- Nachricht
  - Kommunikation & Richtung zwischen Teilnehmern einer Interaktion
  - Aufruf einer Operation oder Senden eines Signals an Empfänger
- Nachrichtenarten
  - Synchron: Sender wartet auf Ausführung der Operation bzw. Verarbeitung des Signals durch Empfänger
  - Asynchron: Sender setzt seine Tätigkeit fort, ohne auf Verarbeitung des Signals durch Empfänger zu warten
  - Operationsaufrufe mit/ohne Rückgabe immer synchron
- Antwort-Nachricht
  - Antwort auf (synchronen) Operationsaufruf, ggf. Rückgabewert
  - Beachte: die Rückgabe kann schon im Aufruf definiert werden; wenn Zeitpunkt der Antwort durch Kontext / Sequenznummer eindeutig, dann kann Antwort-Nachricht weggelassen werden
  - Antwort auf asynchrone Nachrichten durch Aufruf einer Callback-Operation beim Sender, d.h. durch normalen Operationsaufruf
- Optional: Sequenznummer gibt hierarchische Abfolge der Nachrichten an

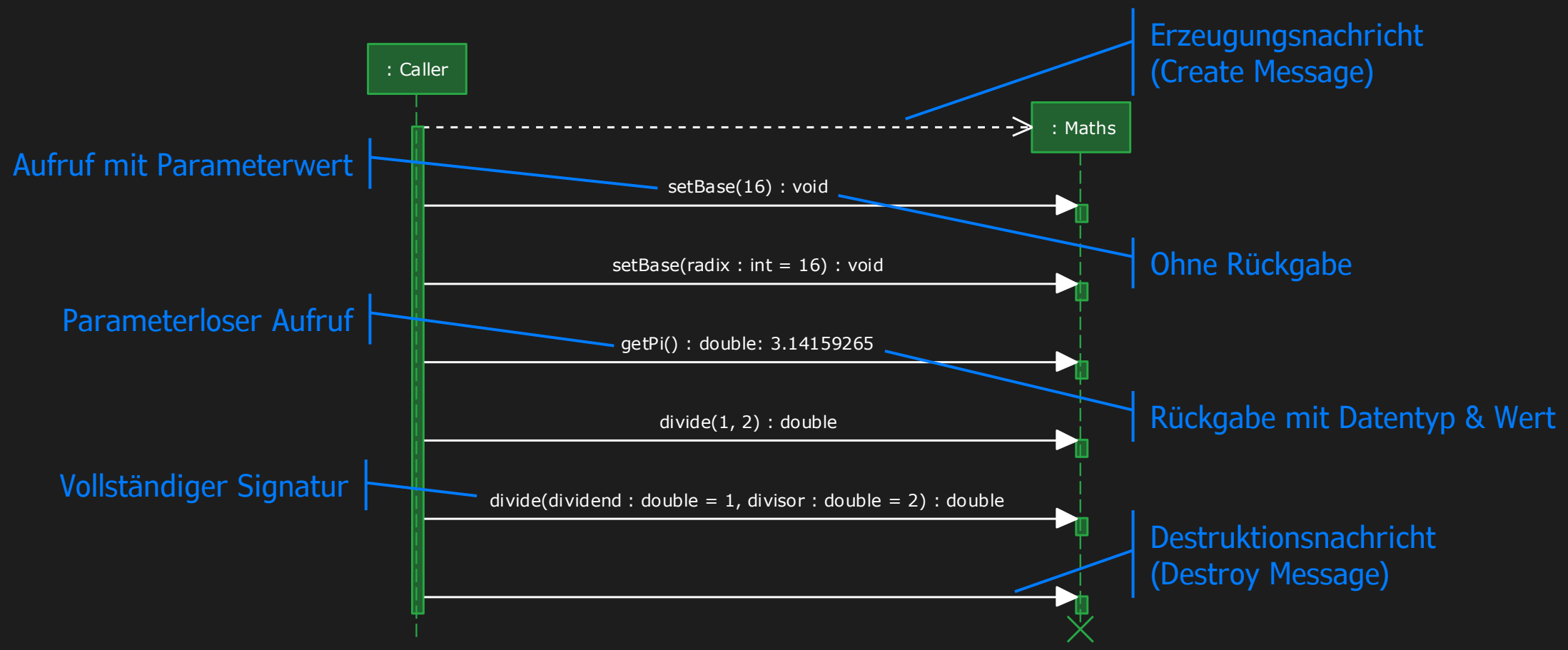


Beispiel: Kartenlesegerät (reduziert)



Beispiel mit Sequenznummern und ohne Antwort-Nachrichten

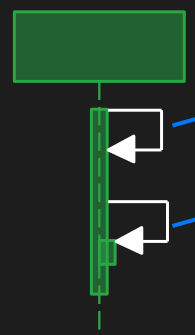
Parameter & Rückgabewerte



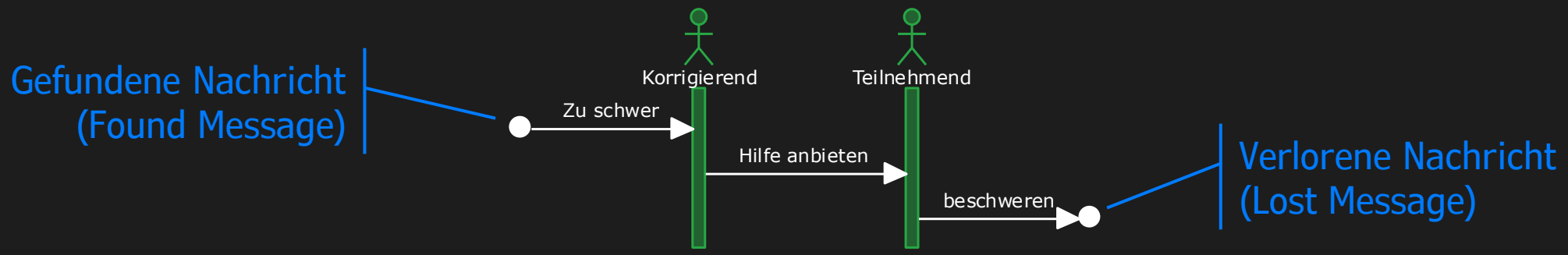
- Parameter & Rückgabewerte können unspezifiziert bleiben (Bindestrich –)
- Unspezifizierte Anzahl & Typ bei Übergabeparametern möglich (Stern *)

Parameter & Rückgabewerte, weitere Nachrichten

- Selbst-Nachricht & rekursive Nachrichten
  - Selbst-Nachricht: gleiche Lebenslinie, gleicher Ausführungsbalken
  - Rekursive Nachricht: gleiche Lebenslinie, neuer Ausführungsbalken

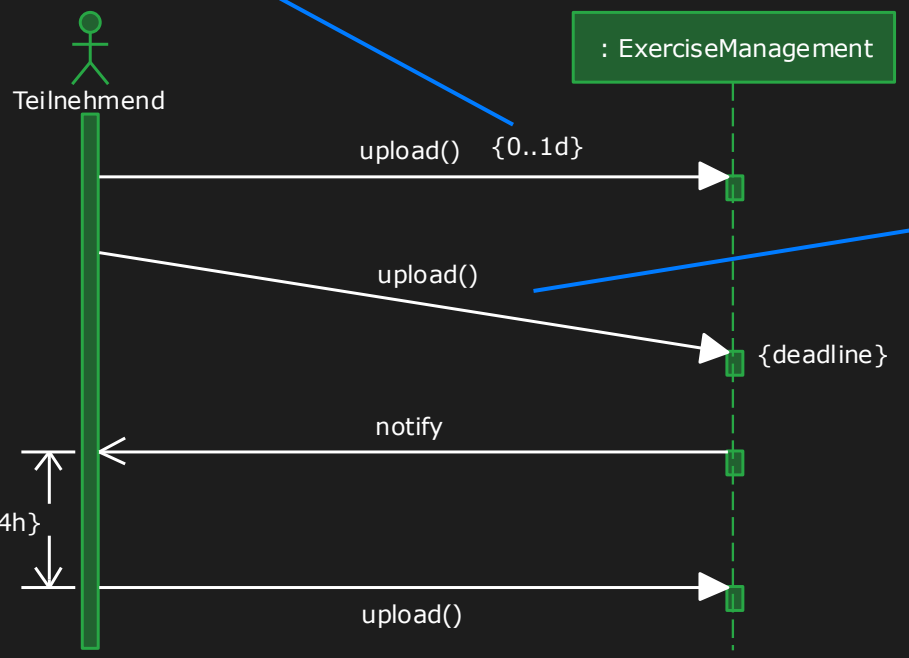


- Verlorene & gefundene Nachrichten
  - Echte Nachrichten, aber Sender bzw. Empfänger zum Entwurfszeitpunkt unbekannt oder außerhalb des Kontexts



Zeitbedingungen

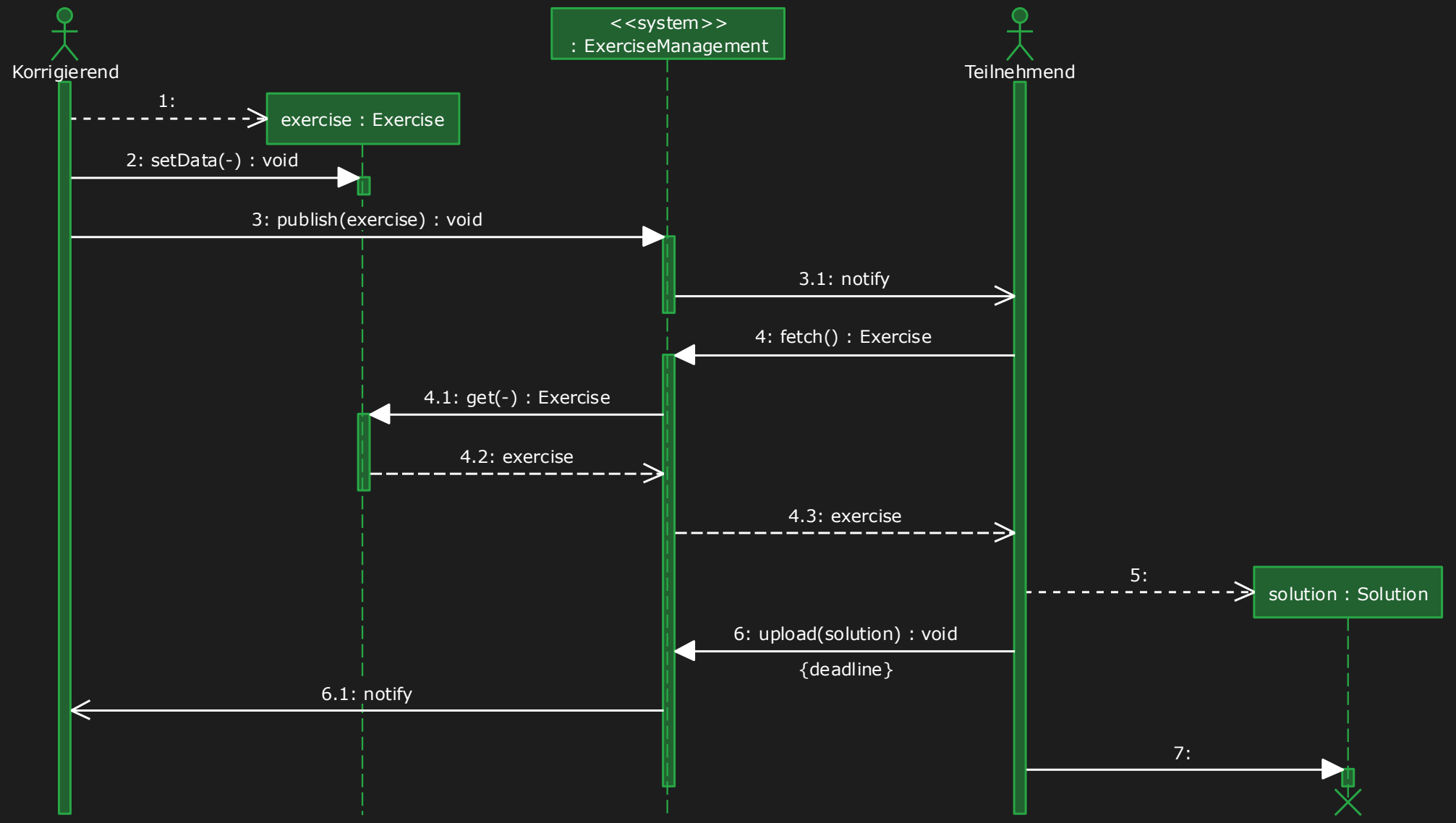
Dauer der Nachricht als Intervall



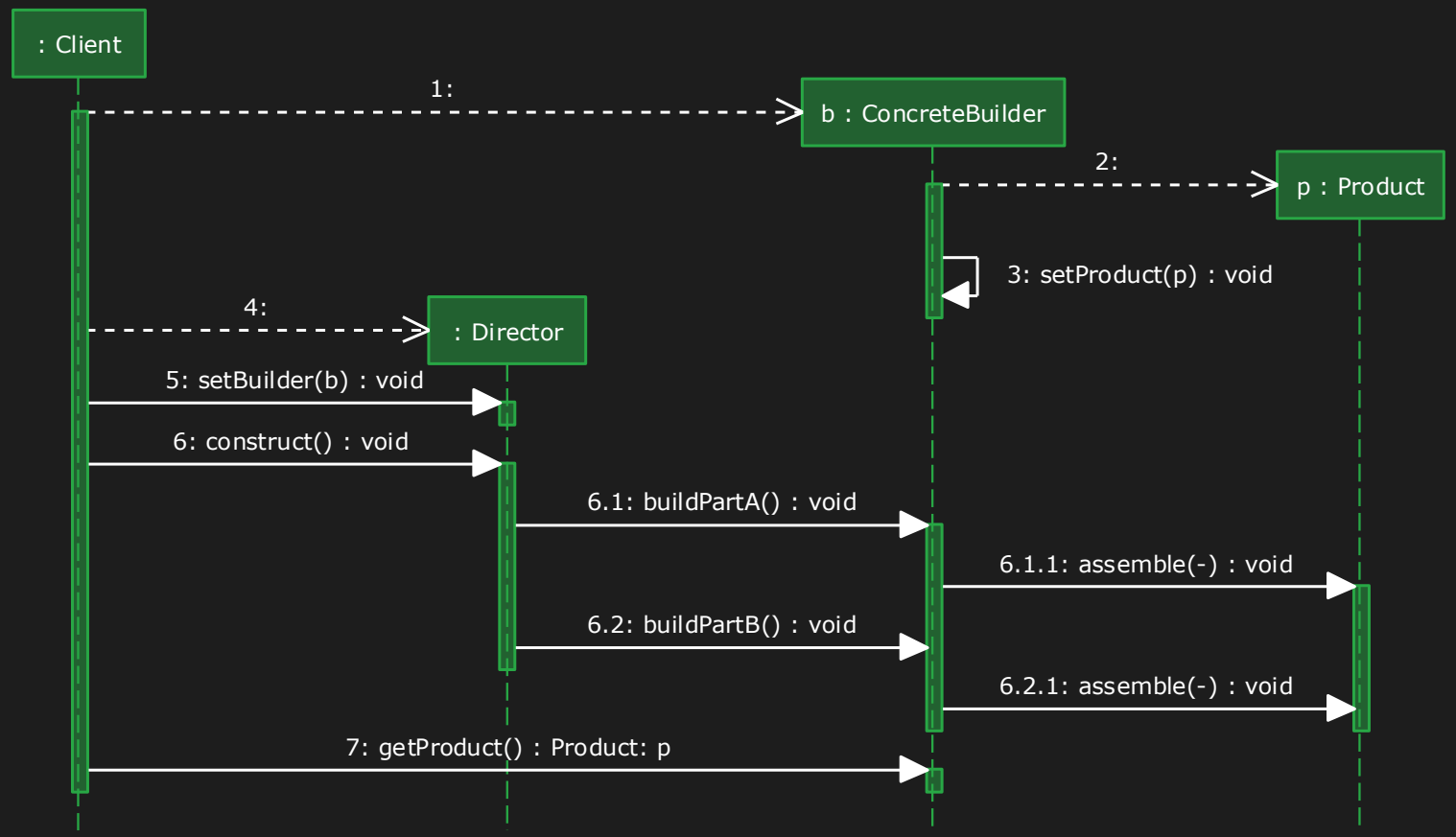
Darstellung einer Nachricht mit »Dauer« & Zeitbedingung

Dauer zwischen Nachrichten

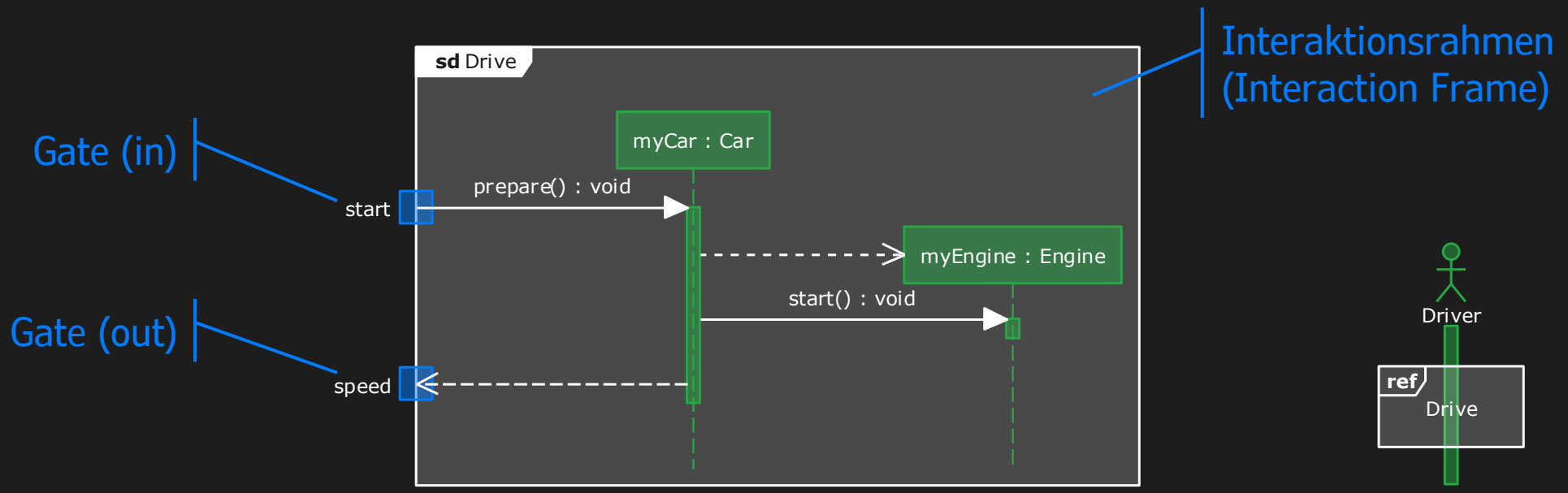
Beispiel: Übungssystem



Entwurfsmuster Builder



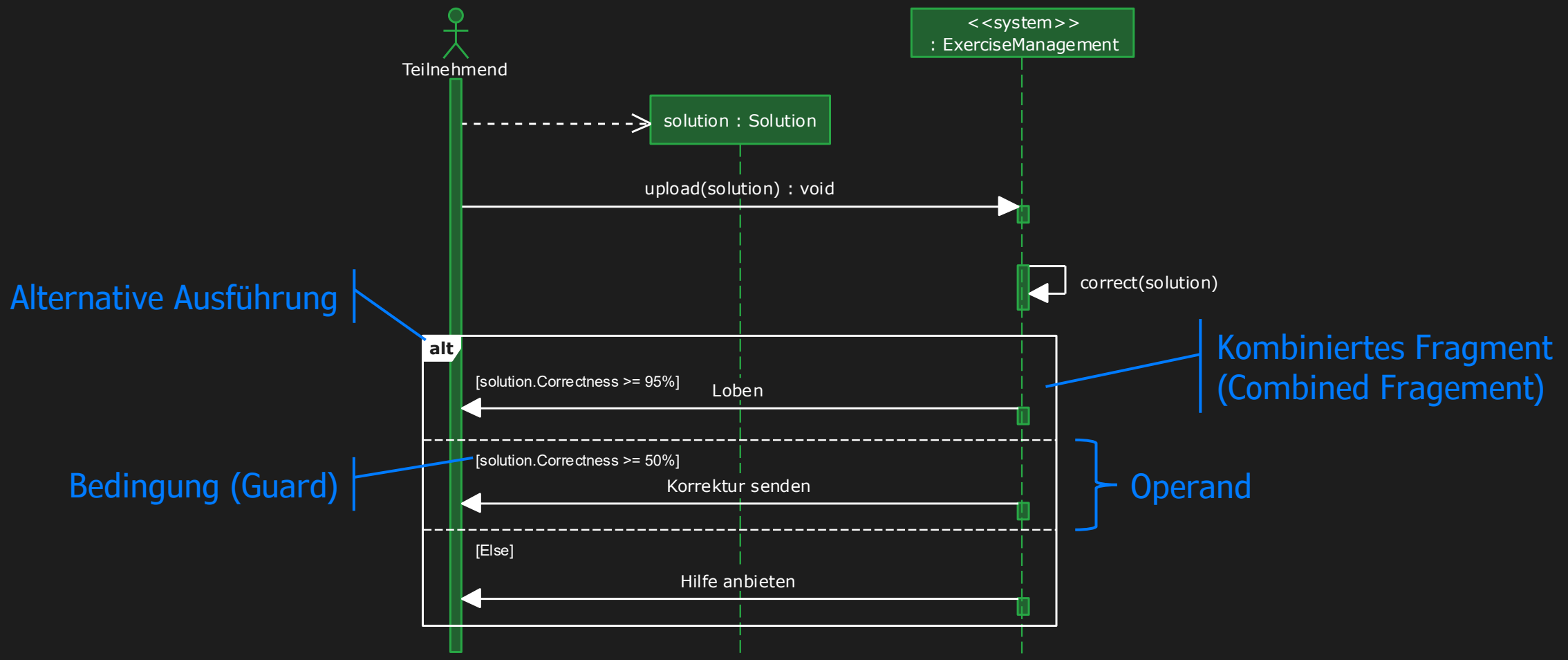
Interaktionsrahmen



Interaktionsreferenz
(Interaction Use)

- Interaktionsrahmen
  - Kapselung & Wiederverwendung
  - Können lokale Attribute (analog zu Klassen) definieren
  - Können Übergabeparameter und einen Rückgabeparameter besitzen
  - Interne Nachrichten dürfen Rahmen nicht verlassen, Austausch von Nachrichten nur über Gates
  - Können in anderen Sequenzdiagrammen über Interaktionsreferenz genutzt werden, Bezeichnung »ref«

Kombiniertes Fragment (1)



Kombiniertes Fragment (2)

Operator	Bedeutung
alt	If-Then-Else-Verzweigung; Wächter definieren die Bedingungen; Fortsetzungen zwischen Fragmenten sind möglich
opt	Optionale Ausführung, wenn Wächterbedingung erfüllt ist
loop	Schleife mit Abbruchbedingung oder Anzahl der Iterationen
par	Parallele Ausführungen; beliebiger Reihenfolge, aber Ordnungsbeziehungen möglich
seq / strict	Schwache / strikte Sequenz: bestimmte Reihenfolge der Nachrichten
neg	Ungültige Nachrichtenfolge, explizite Definition fehlerhafter Sequenzen
critical	Kritischer Bereich: Ausführung darf nicht unterbrochen werden
assert	Sicherstellung von Bedingungen
break	Abbruch der Teilsequenz, wenn Bedingung erfüllt ist

Hinweise

- Sequenzdiagramme werden schnell unübersichtlich
- Auf ein Szenario & wesentliche Objekte beschränken
- Prüfen, welche Alternativen dargestellt werden sollen
- Prüfen, wer die Sequenz startet (Auslöser)
- Teilsequenzen auslagern und durch Interaktionsrahmen darauf verweisen
- Typische Fehler
  - Personen immer als Akteure, nicht als Lebenslinie
  - Akteur wird nicht »erzeugt« oder »destruiert« und hat keine »Operationen«
  - Lebenslinien haben immer einen Typ
  - Nachrichten haben immer eine Richtung
  - Auf asynchrone Nachrichten gibt es keine Antworten